Neurotrack EM — Guía de estudio para la defensa individual (40 %)

> Documento de estudio interno. No es un entregable. Léelo en orden la primera vez y luego usa el índice y el banco de preguntas (sección 20) para repasar.
>
> **Cómo está escrita:** cada tema dice (1) qué es y por qué se hizo así, (2) **en qué archivos está**, (3) **qué código debes saber explicar línea por línea** (con el fragmento real del repositorio) y (4) qué pregunta te pueden hacer. Todo lo que aparece aquí se verificó contra el código del repositorio el 2026-11-11; cuando algo no se pudo comprobar, se dice.
>
> **Convención de rutas:** todas son relativas a la raíz del repo (`C:\Users\Samuel\Desktop\NEUROTRACK_EM`). El alias `@/` en el código significa `src/` (está en `tsconfig.json`: `"@/*": ["./src/*"]`).

Índice

0. Las reglas del juego: cómo es la defensa y cómo responder
1. El proyecto en dos minutos (y los números que debes saber de memoria)
2. Fase 1 — Visión, problema y estado del arte
3. Fase 2 — Propuesta de accesibilidad
4. Fase 3 — Qué se construyó y en qué orden
5. El stack: qué usamos y por qué (tabla de decisiones)
6. Arquitectura: carpetas, capas y el recorrido de un dato
7. Navegación con Expo Router
8. Manejo de estado con Redux Toolkit
9. Capa de datos, sesión segura y manejo de errores
10. Backend y base de datos: Supabase, Postgres, RLS, funciones y triggers
11. Autenticación, correos y consentimiento
12. Lógica de dominio pura y pruebas (Jest)
13. Los módulos de la app, uno por uno
14. Accesibilidad técnica a fondo
15. Rendimiento
16. Decisiones documentadas (D-001 a D-020) para memorizar
17. Calidad del repositorio, Git y reparto del equipo
18. Errores reales que encontramos y cómo se arreglaron
19. Limitaciones conocidas y trabajo futuro
20. Banco de preguntas con respuestas modelo
21. Inspección de código en vivo: ejercicios para practicar
22. Glosario
23. Mapa completo de archivos
24. Plan de estudio y lista de verificación

---

0. Las reglas del juego: cómo es la defensa y cómo responder

0.1 Lo que dice el enunciado (Fase 3, §5)

- **Es el 40 % de la nota.** Después de la presentación, el profesor hace **preguntas técnicas directas a cada integrante, de forma aleatoria e individual**, sobre **cualquier parte del proyecto**: arquitectura de React Native, manejo de estado, consumo de APIs/base de datos, implementación técnica de accesibilidad (`accessibilityRole`, `accessibilityLabel`, manejo de foco, `lineHeight`, `Pressable`) o decisiones documentadas en las Fases 1, 2 y 3.
- **Carácter personal e intransferible:** nadie puede responder, interrumpir ni completar la pregunta de un compañero.
- **Inspección de código en vivo:** el profesor puede pedir que abras el repositorio en tu editor, **ubiques un archivo o componente**, **expliques línea por línea** o **hagas un ajuste en vivo**.

0.2 La escala (qué te da puntos)

Nivel	Qué evalúa el profesor	Puntos
No supo responder	No sabe dónde están los componentes, no explica el estado ni la navegación, ignora cómo se hizo la accesibilidad, o evidencia que no participó	0 %
Respondió parcialmente	Ubica archivos y describe superficialmente qué hace la vista, pero **falla al justificar decisiones, flujo de datos, ciclo de vida de hooks, asincronía o el funcionamiento real de la accesibilidad**	50 %
Respondió perfecto	Argumenta con solvencia las decisiones, domina el código de principio a fin (UI, lógica, datos, accesibilidad) y responde con precisión a las contrapreguntas	100 %

Conclusión práctica: no basta con saber «qué hace». Para el 100 % hay que poder decir **qué hace, dónde está, por qué se hizo así, qué alternativa se descartó y qué pasa si falla**.

0.3 Una fórmula para responder cualquier pregunta técnica

Usa siempre este orden (30–60 segundos):

1. **Respuesta directa** en una frase («Usamos Redux Toolkit para el estado global»).
2. **Dónde está** (`src/store/store.ts`, `src/store/slices/…`).
3. **Cómo funciona** (el mecanismo, con el nombre real de la función).
4. **Por qué** (la razón y la alternativa descartada; casi siempre está en `docs/DECISIONES.md`).
5. **Qué pasa si falla / limitación** (muestra honestidad: eso suma, no resta).

Si no recuerdas un detalle, no inventes: di dónde lo buscarías («está en el slice de actividades; lo abro y se lo muestro»). Con inspección de código en vivo, abrir el archivo correcto ya vale mucho.

0.4 Una advertencia sobre la honestidad

El proyecto declara límites reales (solo Android, sin caché offline, el código de invitación del cuidador lo genera el cliente). **Preséntalos tú antes de que los descubra el profesor.** Un límite declarado con su razón demuestra comprensión; uno descubierto por el profesor parece desconocimiento.

---

1. El proyecto en dos minutos

1.1 El «pitch»

Neurotrack EM («Tu evolución, tu proceso, tu camino») es una app móvil de **seguimiento, rehabilitación y acompañamiento para personas con esclerosis múltiple (EM)** y para quienes las acompañan. **Complementa, no reemplaza,** el seguimiento médico: no diagnostica, no genera alertas clínicas y no usa sensores.

Tiene **tres perfiles**:

- **Paciente:** registra síntomas (calendario), hace actividades de estimulación cognitiva (Memoria, Atención, Lógica) y motriz (Movimiento) con dificultad progresiva y **sin cronómetro**, ve su progreso, recibe recordatorios y mensajes de ánimo, tiene un espacio de apoyo y **decide qué ve su cuidador y su médico**.
- **Cuidador:** se vincula con un código, ve lo que el paciente autorizó, anota observaciones **privadas**, crea recordatorios y envía mensajes de ánimo.
- **Médico tratante (opcional/extra):** ve **solo lectura** un resumen que el paciente autoriza; el acceso **caduca** y el paciente puede retirarlo.

Lo que la hace distinta: **permisos que controla el paciente y que se aplican en la base de datos (RLS), no en la pantalla**, y **accesibilidad como requisito del dominio** (la EM afecta visión, cognición, motricidad y causa fatiga).

1.2 Números que debes saber de memoria

Dato	Valor	Dónde se verifica
SDK de Expo	58 (`expo ~58.0.23`), React Native 0.88.1, React 19.3.0, TypeScript ~6.0.3	`package.json`
Pruebas automatizadas	**124 pruebas en 12 suites** (Jest)	`npm test`
Slices de Redux	**9** (auth, profile, settings, symptoms, activities, progress, caregiver, doctor, reminders)	`src/store/slices/`
Migraciones SQL	**10**	`supabase/migrations/`
Tablas de dominio	symptom_logs, activity_progress, activity_sessions, caregiver_links, caregiver_observations, caregiver_cheers, reminders, doctor_links, claim_attempts	migraciones
Áreas de actividad	**4**: Memoria (5 niveles), Atención (5), Lógica (5), Movimiento (4)	`features/activities/constants.ts`
Rondas por sesión / para subir de nivel	3 / **2 de 3 correctas**	`ROUNDS_PER_SESSION`, `ROUNDS_TO_LEVEL_UP`
Área táctil mínima	**48 dp** (`MinTouchSize`)	`src/theme/spacing.ts`
Escala de texto de la app	normal ×1, grande ×1,15, extra grande ×1,3	`settings-slice.ts`
Contraste	mínimo 4,5:1 texto y 3:1 íconos/bordes; **18 pares validados por script**	`scripts/validar-paleta.mjs`
Requisitos	39: 35 verificados · 1 sin probar (README) · 3 pendientes (demo, defensa, commits)	`docs/REQUISITOS.md`

Dato	Valor	Dónde se verifica
Código de invitación	cuidador: 6 caracteres (alfabeto de 32 sin 0/O/1/I); médico: 8 caracteres, vale 7 días	`caregiver/api.ts`, `doctor_links.sql`
Límite de intentos de código	10 fallos por 15 minutos	`claim_invite`, `claim_doctor_invite`
Referencias del documento	13, IEEE, verificadas	`docs/ENTREGA-FINAL.md`

1.3 Reparto del equipo (para saber qué defender de cada cosa)

Integrante	Módulos (README y documento final)
Samuel	`auth`, `symptoms`, `legal`, `lib/supabase` (síntomas, datos y autenticación)
Amelyn	`caregiver`, `reminders`, `support` (módulo del cuidador)
Mariana	`activities`, `progress` (actividades y progreso)
Extensiones del equipo	`doctor`, `audio`, `settings` (sin dueño asignado: conviene que **cada uno** entienda lo básico)

> **Importante:** el profesor puede preguntar **cualquier parte** a **cualquiera**. Tu módulo se debe dominar al 100 %; el resto, al menos hasta el nivel de «qué hace, dónde está y por qué» de esta guía.

---

2. Fase 1 — Visión, problema y estado del arte

Fuente: `docs/enunciado/NEUROTRACK_EM - FUENTE DE VERDAD.pdf` (el artículo en formato IEEE de las Fases 1 y 2). En el código y en los documentos se cita por sección: §I… §V.

2.1 El problema (§II)

- La EM es una enfermedad neurodegenerativa **muy variable**: cada paciente la vive distinto. Los controles médicos **no permiten monitoreo continuo**; entre cita y cita, pacientes y cuidadores quedan **sin acompañamiento**.
- Los cuidadores notan cambios pero **no saben cómo transmitirlos** al médico, y cargan una **carga significativa**.
- Evidencia citada: la alteración cognitiva está presente en **~41 %** de los pacientes (Askari et al., 2024).

2.2 La solución propuesta (§I y §II-C)

Una app que **complemente** el seguimiento: registro de síntomas con calendario, actividades de estimulación cognitiva/motriz, módulo del cuidador con consentimiento, historial para la próxima consulta, y apoyo emocional. Tres perfiles: paciente, cuidador y **médico (opcional)**.

2.3 Análisis competitivo (§III) — memoriza la tabla

Herramienta	Qué hace	Brecha que encontramos	Cómo la cubre Neurotrack EM
My MS Manager (MSAA)	Registra síntomas, reportes, recordatorios de medicación, noticias	Solo registro; sin rehabilitación ni cuidador	Calendario **más** actividades **más** cuidador con permisos

Herramienta	Qué hace	Brecha que encontramos	Cómo la cubre Neurotrack EM
Floodlight MS (Roche)	Pruebas con sensores del celular (cognición, mano, marcha)	Casi puramente clínica; sin cuidador; desuso con el tiempo	Sin sensores; tono de acompañamiento; apoyo con líneas de ayuda
MSCopilot (Ad Scientiam)	Pruebas funcionales + telerrehabilitación + portal	**Demasiado texto**, genera fatiga; ignora a la familia	Una acción principal por pantalla; instrucciones de una frase; sin cronómetro
MyTherapy	Recordatorios de medicación y diario de síntomas	Genérico, no para EM; sin cuidador ni apoyo	Recordatorios creados por el cuidador, con alternativa visual y háptica
Transversal	—	Casi todas en inglés/francés, para sistemas de salud europeos o norteamericanos	Interfaz en español; **Línea 106 y Emergencias 123 de Colombia**

2.4 Diferenciadores / innovación (§IV-B) y cómo se cumplieron

1. **Tres perfiles en un ecosistema con permisos controlables** → `caregiver_links` y `doctor_links` con RLS; el paciente es el único que acepta.
2. **Cuidador activo** (no solo recibe reportes) → observaciones privadas, recordatorios, mensajes de ánimo.
3. **Sin sensores ni hardware validado** → actividades simples de memoria, atención, lógica y movimiento; `RT-003` verificado (no hay imports de sensores).
4. **Pensada para Latinoamérica** → español, Colombia, Línea 106.
5. **UVP:** «un acompañamiento integral, simple y humano entre consulta y consulta, **sin quitarle el protagonismo al médico tratante**».

2.5 Factibilidad y alcance (§IV-C) — esto decide qué está «dentro» y «fuera»

- **Dentro del MVP:** registro de síntomas, calendario, set reducido de actividades de estimulación, módulo básico del cuidador.
- **Fuera:** biomarcadores digitales, sensores/wearables, validación clínica. El **perfil del médico** se dejó como trabajo futuro, pero **se hizo como extra** (D-016).
- **Riesgos que la propuesta ya anticipaba:** datos de salud sensibles → privacidad desde el diseño; actividades repetitivas → abandono; evaluaciones no validadas clínicamente → dejar claro que **es un apoyo y no un diagnóstico**.

2.6 Una precisión sobre las referencias (por si te preguntan)

Al verificar las referencias del documento final se corrigió la **[3]**: el artículo de Heesen *et al.* sobre salud móvil en EM es del ***Multiple Sclerosis Journal*** (29(14):1709–1720, 2023), **no** del *Journal of Neurology, Neurosurgery & Psychiatry* como decía el artículo de la Fase 1. Además, esa revisión analizó **13 ensayos aleatorizados, la mayoría pilotos**; la cifra «36 estudios con 3091 pacientes» del artículo de la Fase 1 **no corresponde** a ese trabajo. El documento final ya usa la cita y la descripción correctas. Si el profesor compara con la Fase 1 y pregunta por qué cambió, la respuesta es: *«al verificar cada referencia encontramos el error y lo corregimos»*. También se cambió «OMS 2023» por **2022**.

---

3. Fase 2 — Propuesta de accesibilidad (§V del artículo)

La propuesta se basa en **WCAG 2.2 nivel AA** y los principios **POUR** (Perceptible, Operable, Comprensible/Understandable, Robusto), con la justificación de que «una herramienta digital para pacientes neurológicos que no es accesible se convierte en una barrera». Se trabajaron **seis dimensiones**:

Dim.	Promesa de la propuesta	Cómo se implementó (resumen; el detalle técnico está en la sección 14)
V-A Audición	Alternativas visuales y **hápticas** a las alertas sonoras; **subtítulos/transcripción** obligatorios; `accessibilityLiveRegion`	Recordatorios con ícono de check + vibración (`expo-haptics`); guías de Apoyo «Escuchar/Ver» entregadas **como transcripción de texto**; `accessibilityLiveRegion` en 35 lugares
V-B Cognición	Instrucciones cortas, pasos simples, **dificultad progresiva sin que la velocidad sea el único criterio**, poca información por pantalla	Asistente de síntomas de 3 pasos; `InstructionCard` de una frase; niveles por **precisión (2 de 3)**; **sin cronómetro**; `ScreenState`
V-C Motricidad	Botones y áreas táctiles amplias; evitar gestos complejos; actividades motrices a su ritmo	`MinTouchSize = 48`, `hitSlop`; solo toques simples; actividad **Movimiento** por pasos «a tu ritmo»
V-D1/D2 Baja visión	Texto escalable sin recortes ni superposición; botones generosos; un elemento principal por pantalla	`ThemedText` escala `fontSize` y **`lineHeight`**; selector de tamaño de texto; tipografía **Atkinson Hyperlegible Next**
V-D3 Daltonismo	Nada solo por color; evitar rojo-verde; **validar la paleta contra protanopia, deuteranopia y tritanopia**	Severidad con **barras + palabra**; ✓/✕ con texto; `scripts/validar-paleta.mjs` simula las tres
V-D4 Brillo	Modo oscuro/bajo brillo, sin blanco puro sobre negro puro, control independiente del brillo del celular	`DarkColors` (sin `#000`/`#fff` puros); apariencia claro/oscuro/automático **independiente del sistema**; alto contraste opcional
V-D5 Ceguera total	Interfaz navegable con **TalkBack/VoiceOver**, etiquetas descriptivas, orden lógico; **dictado** para síntomas; **versión auditiva** de actividades	`accessibilityRole/Label/Hint/State` en todos los controles; botón **Dictar con voz**; **Lectura en voz alta** con `expo-speech`
V-D6 Factibilidad	Usar las APIs de accesibilidad **ya incluidas** en el framework; priorizar escalado de texto, contraste validado y etiquetado	Todo con props nativas de React Native; sin librerías externas de accesibilidad

> **Punto clave de la defensa:** el enunciado dice que la propuesta del Corte 2 debe estar **«completamente implementada en el código fuente»**. La **matriz trazable** (Tabla 4 del documento final) lista cada mejora, el archivo exacto y la prueba. Debes poder abrir cualquiera de esos archivos.

---

4. Fase 3 — Qué se construyó y en qué orden

El plan se organizó en tareas (T0…T11) pensando en que **siempre hubiera algo funcionando**. Es la «historia» del proyecto: sirve para explicar *por qué* está hecho así.

Tarea	Qué se hizo	Requisitos	Decisión relacionada
T0 Base	Tokens de tema accesibles (`src/theme/`), navegación por perfil con Expo Router, estados carga/error/vacío (`ScreenState`), store Redux	RT-001, RT-006, RNF-001..009	D-001, D-005
T0.5 Datos + Auth	Proyecto Supabase, cliente, registro/login, **sesión en almacenamiento seguro**, manejo de errores y offline	RF-011, RNF-012	D-002
T1 Síntomas	Asistente de 3 pasos + calendario del mes, persistidos en `symptom_logs` con RLS	RF-001, RF-002	—
T2 Actividades	Memoria, Atención, Movimiento, niveles por precisión; (después Lógica)	RF-003, RF-004	D-009, D-018
T3 Progreso	Historial y comparación actual vs. anterior (semana/mes/3 meses), gráfico y tabla	RF-005	—
T4 Cuidador	Vinculación con consentimiento, observaciones privadas, recordatorios, mensajes de ánimo	RF-006, RF-008, RF-012	D-010
T5 Apoyo	Contactos de crisis (106/123), guías con transcripción	RF-009, RF-010	—
T6 Accesibilidad + pruebas	Auditoría de código, pantalla Accesibilidad, **modo oscuro real**, háptica en actividades, **Jest**	RNF-*	D-004, D-011, D-012
T7 Voz	Dictado (teclado del sistema) y «Lectura en voz alta» (`expo-speech`)	REC-001, REC-002	D-013
T7b Correos	Confirmar cuenta y recuperar contraseña con página web en Vercel, textos legales y consentimiento	RF-011	D-014, D-015
T8/T9 Entrega	`ENTREGA-FINAL.md`, `README.md`	ENT-002, ENT-003	D-006
T10 Médico	Perfil de solo lectura con permisos, caducidad y resumen comparativo	REC-003	D-016
T11 Cierre	Contraste en oscuro (`primary`), gráfico de Progreso seleccionable, actividad Lógica, nombre del paciente al cuidador	RNF-003, RF-003	D-017, D-018, D-019

Cómo se verificó: `tsc --noEmit` y `eslint` limpios; **124 pruebas Jest**; pruebas manuales en **emulador Android** y en **celular real con Expo Go**; capturas en `artifacts/` (t0…t11, `talkback/`); y una **auditoría de accesibilidad de código** con un agente revisor.

Una frase para el profesor: *«Construimos primero la base y la capa de datos, luego cada módulo de forma vertical (pantalla + estado + datos + permisos + prueba), y dejamos la accesibilidad como requisito transversal verificado en dispositivo real.»*

---

5. El stack: qué usamos y por qué

> Esta es la pregunta más probable: **«¿qué usaron para X y por qué?»**. Aprende la columna «Por qué».

Capa	Tecnología	Por qué	Alternativa descartada
Framework móvil	**Expo + React Native** (SDK 58)	Un solo código para Android/iOS; el entorno del equipo ya estaba andamiado; ejecuta en **Expo Go** sin compilar nativo	React Native «puro» (más configuración nativa)

Capa	Tecnología	Por qué	Alternativa descartada
Lenguaje	**TypeScript estricto** (`"strict": true`)	Tipos para modelos y errores; los slices y las APIs están tipados	JavaScript
Navegación	**Expo Router** (rutas por archivos en `src/app/`)	Es el estándar de Expo; grupos de rutas por perfil; **rutas tipadas** (`typedRoutes`)	React Navigation manual
Estado global	**Redux Toolkit** + `react-redux`	Exigido/permitido por el enunciado («Redux Toolkit o Context»); maneja **asincronía** con `createAsyncThunk`, trazabilidad y reseteo al cerrar sesión (D-005)	Context API + hooks (se queda corto para asincronía, caché y reseteo)
Backend	**Supabase** (Auth + Postgres + **Row Level Security**), capa gratuita	Sincronización real entre dispositivos (el enunciado prohíbe *mocks*) y **permisos por fila** para «acceso con autorización» (D-002)	Firebase; solo local (no sincroniza)
Cliente backend	`@supabase/supabase-js`	SDK oficial; maneja sesión, refresco de token y RPC	`fetch` manual
Sesión segura	`expo-secure-store` + `aes-js` + `AsyncStorage`	El JWT supera el límite de ~2 KB de SecureStore en Android → patrón `LargeSecureStore`	Guardar la sesión en AsyncStorage sin cifrar
Aleatoriedad	`react-native-get-random-values`	`crypto.getRandomValues` para la clave AES y los códigos de invitación	`Math.random` (no es criptográfico)
Háptica	`expo-haptics`	Vibración de éxito/error (RNF-009: nunca solo un canal sensorial)	—
Voz	`expo-speech`	Texto a voz **sin conexión y sin enviar nada a un servicio** (D-013)	Reconocedor de voz propio (no existe en SDK 58 / exige build nativa)
Tipografía	**Atkinson Hyperlegible Next** (`@expo-google-fonts`)	Diseñada para **baja visión** (Braille Institute)	Fuente del sistema
Íconos	`@expo/vector-icons` (Material Icons)	Íconos reconocibles; complementan el color (RNF-006)	—
Seguridad de datos	**RLS + funciones `security definer` + triggers** en Postgres	El control de permisos vive en el servidor, no en la interfaz	Validar solo en el cliente (inseguro)
Pruebas	**Jest** con `jest-expo`	Lógica pura testeable sin UI (D-004)	Sin pruebas
Calidad	`tsc --noEmit` + **ESLint** (`eslint-config-expo`)	Regla de «terminado»: sin errores de tipos ni de lint	—
Web de correos	Página **estática** en **Vercel** (`web-auth/`)	El enlace del correo debe funcionar en **cualquier dispositivo** (D-014)	Enlace profundo a la app (falla con PKCE/Expo Go)
Correo	**SMTP propio (Gmail)** en Supabase	El correo integrado de Supabase solo entrega a miembros de la organización	Correo integrado
Diseño	**Google Stitch** (exports) + Material 3	Paleta y componentes de referencia; adaptados por el equipo	—

Configuración relevante de `app.json`: `scheme: "neurotrackem"`, `userInterfaceStyle: "automatic"`, plugins `expo-router`, `expo-splash-screen`, `expo-secure-store`, y los experimentos **`typedRoutes: true`** (rutas con tipo) y **`reactCompiler: true`** (React Compiler: memoiza automáticamente componentes).

Por qué «Expo Go» y no una build propia: el equipo verificó en emulador y celular con Expo Go; por eso se evitó cualquier librería con código nativo que Expo Go no incluya (de ahí el dictado con el

teclado del sistema en vez de un reconocedor propio).

---

6. Arquitectura: carpetas, capas y el recorrido de un dato

6.1 Estructura del repositorio

```
NEUROTRACK_EM/
├─ src/
│  ├─ app/            ← PANTALLAS y rutas (Expo Router). Cada archivo es una ruta.
│  ├─ components/     ← Componentes reutilizables sin lógica de negocio (ThemedText, ScreenState,
TextField…)
│  ├─ features/       ← UN MÓDULO POR FUNCIONALIDAD: api.ts + logic.ts + types.ts + constants +
componentes propios
│  ├─ store/          ← Redux: store.ts, actions.ts, hooks.ts y slices/
│  ├─ theme/          ← Tokens: colores (claro/oscuro/alto contraste), tipografía, espaciado
│  ├─ lib/supabase.ts ← El cliente de Supabase + sesión cifrada
│  └─ hooks/          ← use-color-scheme
├─ supabase/
│  ├─ migrations/     ← 10 archivos SQL (tablas, RLS, funciones, triggers)
│  ├─ seed/           ← datos de demostración
│  └─ email-templates/← plantillas HTML de los correos
├─ web-auth/          ← página estática (Vercel) para confirmar correo y restablecer contraseña
├─ scripts/validar-paleta.mjs ← valida contraste y daltonismo de la paleta
├─ docs/              ← REQUISITOS, DECISIONES, ENTREGA-FINAL, STATUS, ENTORNO, enunciado/, diseño…
├─ artifacts/         ← capturas de verificación por módulo (t0…t11, talkback)
└─ README.md, app.json, package.json, tsconfig.json, eslint.config.js
```

Regla del proyecto (CLAUDE.md/AGENTS.md): *las rutas viven en `src/app/`; el código que no es una ruta (componentes, hooks, utilidades) va fuera de `src/app/`*. Por eso `src/components`, `src/features`, `src/store`, `src/theme` están separados.

6.2 Las capas (de arriba hacia abajo)

```
 PANTALLA (src/app/…tsx)           ← solo UI: lee del store con useAppSelector y despacha
acciones/thunks
      │  dispatch(thunk)  /  useAppSelector(...)
      ▼
 STORE (src/store/slices/*.ts)     ← estado global + createAsyncThunk (orquesta la llamada y los
estados cargando/error)
      │  llama a…
      ▼
 FEATURE api.ts (src/features/X/api.ts) ← único lugar que habla con Supabase; traduce errores a
español por clase
      │  supabase.from(...) / supabase.rpc(...)
      ▼
 SUPABASE (src/lib/supabase.ts)    ← cliente con sesión cifrada
      │  HTTPS
      ▼
 POSTGRES + RLS + funciones        ← la seguridad real (supabase/migrations/*.sql)

 En paralelo:  features/X/logic.ts → REGLAS PURAS (sin UI, sin red) que usan los slices y las
pantallas, y que se prueban con Jest.
```

Por qué esta separación (la respuesta que esperan):

- La **pantalla** no sabe de red: si cambia el backend solo se toca `api.ts`.

- El **slice** concentra los estados asíncronos (`cargando`, `listo`, `error`) y los traduce a la UI con `ScreenState`.
- La **lógica pura** (`logic.ts`) no depende de React ni de Supabase, así que se puede **probar con Jest** (124 pruebas).
- La **seguridad** no está en la app sino en Postgres: aunque alguien modifique la app, **RLS** impide leer datos ajenos.

> **Excepciones reales que debes conocer** (por si el profesor busca imports de Supabase en pantallas): solo `src/app/_layout.tsx` importa `supabase` (para escuchar `onAuthStateChange` y detectar `SIGNED_OUT`), y `src/app/(auth)/acceder.tsx` llama a `resendConfirmationEmail` de `features/auth/api.ts` (reenviar el correo de confirmación). Todo lo demás pasa por thunks.

6.3 Un módulo por dentro (patrón que se repite)

Ejemplo `src/features/symptoms/`:

Archivo	Responsabilidad
`types.ts`	Tipos: `SymptomType`, `Severity`, `SymptomLog`, `SymptomsError`
`constants.ts`	Listas de opciones (tipos de síntoma, severidades, íconos)
`api.ts`	`createSymptomLog`, `listSymptomLogsForMonth`, `listSymptomLogsBetween`, `listSharedSymptomLogs` + traducción de errores
`calendar.ts`	**Lógica pura**: `buildMonthGrid`, `leadingBlankCount`, `logsForIsoDate`, `isoDateOfDate`
`summary.ts`	Resumen en texto de los síntomas (para compartir)
`wizard-header.tsx`, `wizard-actions.tsx`	Componentes de UI del asistente

Los demás módulos (`activities`, `progress`, `caregiver`, `reminders`, `doctor`, `support`, `audio`…) siguen el mismo molde.

6.4 El recorrido completo de un dato: «registrar un síntoma»

Es **el ejemplo que mejor demuestra toda la arquitectura**. Aprende cada paso con su archivo:

1. **Paso 1** (`src/app/registrar-sintoma/paso-1.tsx`): el paciente toca «Fatiga». La pantalla ejecuta `dispatch(draftSymptomTypeSet('fatiga'))`. Esto **no llama a la red**: guarda el *borrador* en `state.symptoms.draft`.
2. **Paso 2** (`paso-2.tsx`): elige severidad → `dispatch(draftSeveritySet('moderado'))`.
3. **Paso 3** (`paso-3.tsx`): escribe la nota (opcional, máx. 300 caracteres) → `dispatch(draftNoteSet(texto))` y pulsa **Guardar**: se ejecuta `guardar()`:

```
async function guardar() {
  const result = await dispatch(submitSymptomLog());
  if (submitSymptomLog.fulfilled.match(result)) {
    dispatch(draftReset());
    router.dismissTo('/(paciente)/calendario');
  }
}
```

- `dispatch(submitSymptomLog())` devuelve una **promesa** con la acción final (`fulfilled` o `rejected`).

- `submitSymptomLog.fulfilled.match(result)` comprueba que **salió bien** antes de limpiar el borrador y volver al calendario.

4. **El thunk** (`src/store/slices/symptoms-slice.ts`, `submitSymptomLog`): lee del store el borrador y el `userId` de la sesión (`getState().auth.session?.userId`), **valida** que haya tipo y severidad, y llama a `createSymptomLog(patientId, {...})`. Si falta algo hace `rejectWithValue({ errorClass: 'validacion', ... })`.
5. **La API** (`src/features/symptoms/api.ts`, `createSymptomLog`): `supabase.from('symptom_logs').insert({...}).select(...).single()`. **Importante:** manda `occurred_at: new Date().toISOString()` (reloj del dispositivo, no `now()` del servidor) para que el registro caiga en el mismo «hoy» que ve el paciente en el calendario.
6. **Postgres** (`20261108000000_symptom_logs.sql`): la política `symptom_logs_insert_own` hace `with check (auth.uid() = patient_id)`: solo deja insertar si el `patient_id` es **el usuario autenticado**.
7. **De vuelta:** el thunk entra al reducer `submitSymptomLog.fulfilled`, que **agrega el registro al mes en caché** (`state.monthsByKey[key].logs`) si ese mes ya estaba cargado. El calendario (que sigue montado debajo del modal) se **re-renderiza solo** porque lee ese estado con `useAppSelector`.

> **Detalle fino que luce bien explicar:** el calendario pide los datos con `useEffect(() => { dispatch(fetchMonthLogs({ year, month })); }, [dispatch, year, month])`. Solo se vuelve a pedir cuando **cambia el mes**, no cuando se guarda un registro: por eso el reducer `fulfilled` actualiza la caché.

---

7. Navegación con Expo Router

7.1 Qué es y cómo funciona

Expo Router usa **el sistema de archivos como mapa de rutas**: cada archivo dentro de `src/app/` es una pantalla; las carpetas son segmentos de ruta; los archivos `_layout.tsx` definen **cómo se navega** entre sus hijos (pila, pestañas, modal). Los paréntesis `(paciente)` crean un **grupo de rutas**: **no aparece en la URL**, solo organiza.

7.2 El mapa de rutas (qué archivo es cada pantalla)

Ruta / grupo	Archivo(s)	Tipo de navegación
/ (selección de perfil)	`src/app/index.tsx`	Pantalla raíz
Raíz	`src/app/_layout.tsx`	`Stack` + proveedor de Redux + arranque
`(auth)`	`(auth)/_layout.tsx`, `acceder.tsx`, `recuperar.tsx`	Pila de acceso
`(paciente)`	`(paciente)/_layout.tsx` + `inicio`, `calendario`, `actividades`, `progreso`, `apoyo`	**Pestañas nativas** (`NativeTabs`)
`(cuidador)`	`(cuidador)/_layout.tsx` + `inicio`, `acompanar`, `observaciones`, `recordatorios`, `apoyo`	**Pestañas nativas**
`(medico)`	`(medico)/_layout.tsx` + `pacientes`, `vincular-paciente`, `paciente/[id]`	**Pila sin pestañas** (solo consulta)
Asistente de síntoma	`registrar-sintoma/_layout.tsx` + `paso-1/2/3`	**Modal** (`presentation: 'modal'`)
Actividad	`actividad/_layout.tsx`, `actividad/[area].tsx`	Ruta **dinámica** `[area]`
Apoyo (detalle)	`apoyo/_layout.tsx`, `apoyo/[id].tsx`	Ruta dinámica `[id]`

Ruta / grupo	Archivo(s)	Tipo de navegación
Legal	`legal/[doc].tsx`	Ruta dinámica (`terminos` / `privacidad`)
Otras	`cuenta`, `accesibilidad`, `acerca`, `mi-medico`, `permisos`, `vincularme`, `nueva-observacion`, `correo/confirmar`, `correo/restablecer`	Pantallas sueltas

7.3 El flujo de arranque y la redirección por perfil

1. `src/app/_layout.tsx` (`RootLayout`) envuelve todo en `<StoreProvider store={store}>` y renderiza `AppShell`.
2. `AppShell` hace en un `useEffect`: `dispatch(bootstrapSession())` (lee la sesión guardada), `dispatch(hydrateSettings())` (lee las preferencias) y se suscribe a `supabase.auth.onAuthStateChange` para reaccionar a `SIGNED_OUT` con `sessionEnded()` + `resetAll()`.
3. Mantiene el **splash** visible hasta que haya fuentes cargadas, sesión resuelta y preferencias hidratadas:

```
const ready = fontsLoaded && authStatus !== 'bootstrapping' && settingsStatus;
useEffect(() => { if (ready) SplashScreen.hide(); }, [ready]);
if (!ready) return null;
return <Stack screenOptions={{ headerShown: false }} />;
```

Por qué: así no se «parpadea» una pantalla equivocada (p. ej. login) mientras se averigua si ya había sesión. 4. `src/app/index.tsx` (selección de perfil) decide a dónde ir:

```
if (selected && authStatus !== 'authenticated') return <Redirect href="/(auth)/acceder" />;
if (selected) return <Redirect href={PROFILE_HOME[selected]} />;
```

- Sin sesión → manda a `(auth)/acceder`; con sesión → manda al inicio del perfil con el diccionario `PROFILE_HOME` de `src/features/profile/constants.ts` (`paciente → /(paciente)/inicio`, `cuidador → /(cuidador)/inicio`, `medico → /(medico)/pacientes`). **Un solo lugar** para no repetir ternarios por perfil.

5. El perfil elegido vive en `state.profile.selected` (slice `profile`). **No está atado a la cuenta**: se elige al abrir la app (decisión D-016 lo menciona como consecuencia).

7.4 Pestañas nativas (`NativeTabs`)

`src/app/(paciente)/_layout.tsx`:

```
<NativeTabs
  backgroundColor={colors.surface}
  indicatorColor={colors.secondaryContainer}
  iconColor={{ default: colors.onSurfaceVariant, selected: colors.primary }}
  labelStyle={{ default: {...}, selected: { color: colors.primary, fontWeight: 'bold' } }}>
  <NativeTabs.Trigger name="inicio"> … </NativeTabs.Trigger>
  …
</NativeTabs>
```

- Cada `NativeTabs.Trigger name="…"` apunta al **archivo** con ese nombre en la carpeta del grupo.
- **Bug real que se encontró** (To): al inicio la barra no heredaba los colores del tema y se veía en gris «Material You»; se arregló pasando `backgroundColor`, `iconColor`, `labelStyle` e `indicatorColor` desde `useThemeColors()`.

- **Por qué nativas:** la barra la dibuja el sistema (mejor integración con TalkBack, que ya sabe leerlas).

7.5 Modales y navegación imperativa

- `registrar-sintoma/_layout.tsx`: `<Stack screenOptions={{ headerShown: false, presentation: 'modal' }} />` → el asistente se abre **encima** de las pestañas.
- Navegar: `router.push('/registrar-sintoma/paso-2')`, `router.back()`, `router.replace('/')`, `router.dismissTo('/(paciente)/calendario')` (cierra el modal y vuelve a una ruta concreta), `router.dismissAll()`.
- **Parámetros de ruta:** `router.push({ pathname: '/legal/[doc]', params: { doc: 'terminos' } });` en la pantalla destino se leen con `useLocalSearchParams`.
- **`<Redirect href=…/>`** se usa en **render** para guardar rutas (p. ej. `paso-3` redirige a `paso-1` si el borrador está incompleto: `if (!symptomType || !severity) return <Redirect href="/registrar-sintoma/paso-1" />`).

7.6 Preguntas típicas de esta sección

- *«¿Qué diferencia hay entre `router.push` y `router.replace`?»* `push` apila la pantalla (se puede volver); `replace` reemplaza la actual (no se vuelve a ella; se usa tras cerrar sesión).
- *«¿Cómo sabe la app a qué pantalla ir al abrir?»* Splash hasta que `bootstrapSession` termina; `index.tsx` redirige según `profile.selected` y `auth.status`.
- *«¿Por qué los grupos `(paciente)/(cuidador)/(medico)`?»* Para separar la navegación de cada perfil sin que el nombre del grupo aparezca en la ruta, y para que cada uno tenga su propio `_layout` (pestañas o pila).
- *«¿Cómo protegen las rutas?»* Con `Redirect` según `auth.status`; **la protección real de los datos no depende de la ruta, sino de RLS** (si alguien llegara a una pantalla ajena, Postgres no le devolvería filas).

---

8. Manejo de estado con Redux Toolkit

> **La pregunta literal que pueden hacer: «¿Qué usaron para el manejo de estado, Redux o qué?» Respuesta:** *«Redux Toolkit, con `react-redux`. Hay un store con 9 slices tipados; todo lo remoto se hace con `createAsyncThunk` y se refleja en estados `idle | cargando | listo | error`. Está en `src/store/`. Lo elegimos (D-005) porque el enunciado lo pide como opción y porque la app tiene varios estados asíncronos —sesión, vínculos, borradores, caché por mes—, donde Context se queda corto.»*

8.1 Conceptos que debes dominar (con definición corta)

Concepto	Qué es	Dónde se ve en el repo
Store	El objeto único que guarda **todo el estado global**	`src/store/store.ts` → `configureStore`
Slice	Un «trozo» del estado + sus reducers + sus acciones	`src/store/slices/*.ts` → `createSlice`
Reducer	Función pura `(estado, acción) → nuevo estado`	`reducers: { … }` dentro de cada slice
Action	Un objeto `{ type, payload }` que describe «qué pasó»	`dispatch(draftSymptomTypeSet('fatiga'))`

Concepto	Qué es	Dónde se ve en el repo
Immer	Permite *escribir* `state.x = y` en un reducer; por debajo genera un estado nuevo **inmutable**	Por eso los reducers «mutan» sin romper la inmutabilidad
Selector	Lee una parte del estado	`useAppSelector((state) => state.settings.colorScheme)`
Thunk	Una función asíncrona que se despacha (para llamar a la red)	`createAsyncThunk`
`extraReducers`	Reducers que responden a acciones **de otro origen** (p. ej. los estados de un thunk)	`builder.addCase(thunk.pending/fulfilled/rejected, …)`

8.2 `src/store/store.ts` explicado línea por línea

```
const appReducer = combineReducers({
  auth: authReducer, profile: profileReducer, settings: settingsReducer,
  symptoms: symptomsReducer, activities: activitiesReducer, progress: progressReducer,
  caregiver: caregiverReducer, doctor: doctorReducer, reminders: remindersReducer,
});
```

- `combineReducers` junta los **9 slices**; el estado global queda `{ auth, profile, settings, symptoms, … }`.

```
const rootReducer: typeof appReducer = (state, action) => {
  if (resetAll.match(action) && state) {
    return appReducer({ settings: state.settings, auth: state.auth } as typeof state, action);
  }
  return appReducer(state, action);
};
```

- **`resetAll`** (definida en `src/store/actions.ts` con `createAction('app/resetAll')`) se despacha **al terminar la sesión** (cerrar sesión, borrar cuenta, sesión expirada).
- Al detectarla, el reducer raíz **reinicia todos los slices menos `settings` y `auth`** (pasa a `appReducer` un estado que solo conserva esos dos). **Por qué:** son **datos de salud**; la siguiente persona que use el teléfono **no debe ver nada** de la anterior. Las preferencias de accesibilidad se conservan porque son **del dispositivo**, no de la cuenta.

```
export const store = configureStore({ reducer: rootReducer });
let lastPersistedSettings = store.getState().settings;
store.subscribe(() => {
  const settings = store.getState().settings;
  if (settings !== lastPersistedSettings) { lastPersistedSettings = settings;
saveSettings(settings); }
});
export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;
```

- `store.subscribe` **guarda las preferencias en disco cada vez que cambian** (comparando por referencia: Redux Toolkit/Immer solo crea un objeto nuevo si algo cambió). `saveSettings` está en `src/features/settings/persistence.ts` (AsyncStorage, **sin cifrar**: son preferencias, no salud).
- `RootState` y `AppDispatch` son los **tipos** que usan los hooks tipados.

8.3 Hooks tipados (`src/store/hooks.ts`)

```
export const useAppDispatch = () => useDispatch<AppDispatch>();
export const useAppSelector: TypedUseSelectorHook<RootState> = useSelector;
```

Por qué: así `dispatch` acepta thunks con tipos correctos y `useAppSelector` autocompleta el estado. En toda la app se usan estos dos, **nunca** `useDispatch`/`useSelector` «crudos».

8.4 Los 9 slices (qué guarda cada uno)

Slice (archivo)	Qué guarda	Thunks / reducers principales
`auth-slice.ts`	`status` (`bootstrapping/unauthenticated/authenticating/authenticated`), `session`, `error`, `pendingConfirmationEmail`, `linkFlow`	`bootstrapSession`, `login`, `register`, `logout`, `removeAccount`, `openAuthLink`, `changePasswordFromLink`
`profile-slice.ts`	`selected: 'paciente' | 'cuidador' | 'medico' | null`	`profileSelected`, `profileCleared`
`settings-slice.ts`	Preferencias: `colorScheme`, `highContrast`, `reduceMotion`, `textScale`, `audioGuidance`, `_hydrated`	`hydrateSettings`, `colorSchemeChanged`, `highContrastToggled`, `textScaleChanged`, `settingsReset`…
`symptoms-slice.ts`	`draft` del asistente, `submitStatus`, **caché por mes** `monthsByKey`	`fetchMonthLogs`, `submitSymptomLog`, `draft*`
`activities-slice.ts`	`progress` por área (niveles), `session` (máquina de estados de la sesión), `saveStatus`	`fetchAreaProgress`, `persistAreaLevel`, `recordActivitySession`, reducers de cada actividad
`progress-slice.ts`	`period`, `referenceISO`, `view` (gráfico/tabla), datos del rango	`fetchProgressData`, `periodChanged`, `referenceShifted`, `viewToggled`
`caregiver-slice.ts`	Lado paciente (`myInvite`, `incomingRequests`, `myCaregivers`) y lado cuidador (`myLinksAsCaregiver`, `claimStatus`, observaciones, mensajes)	`fetchPatientCaregiverData`, `claimPatientInviteCode`, `respondToIncomingRequest`, `revokeMyCaregiver`, `addObservation`, `sendCheerMessage`…
`doctor-slice.ts`	Vínculos del médico, `claimStatus`, **`summary`** y `summaryRequestId`	`fetchMyPatients`, `claimPatientCode`, `fetchPatientSummary`, `revokeMyDoctor`…
`reminders-slice.ts`	Recordatorios del paciente	`fetchReminders`, `addReminder`, `completeReminder`

8.5 `createAsyncThunk`: anatomía (la pieza clave para la defensa)

Un thunk genera **tres acciones** automáticamente: `pending` (arrancó), `fulfilled` (salió bien, con el valor devuelto) y `rejected` (falló, con el valor de `rejectWithValue`).

Ejemplo real — `fetchMonthLogs` en `symptoms-slice.ts`:

```
export const fetchMonthLogs = createAsyncThunk<
  { key: MonthKey; logs: SymptomLog[] },                // tipo del valor si sale bien
  { year: number; month: number },                      // tipo del argumento
  { state: RootState; rejectValue: SymptomsError }      // acceso al estado + tipo del error
>('symptoms/fetchMonthLogs', async ({ year, month }, { getState, rejectWithValue }) => {
  const patientId = getState().auth.session?.userId;
  if (!patientId) return rejectWithValue({ errorClass: 'inesperado', message: 'No hay sesión activa'
});
  try {
    const logs = await listSymptomLogsForMonth(patientId, year, month);
    return { key: monthKey(year, month), logs };
  } catch (error) {
    return rejectWithValue(error as SymptomsError);
  }
});
```

Y los reducers que responden (`extraReducers`):

```
.addCase(fetchMonthLogs.pending,   (state, action) => { /* status = 'cargando', conserva los logs
previos */ })
.addCase(fetchMonthLogs.fulfilled, (state, action) => { /* status = 'listo', guarda los logs */ })
.addCase(fetchMonthLogs.rejected,  (state, action) => { /* status = 'error', guarda el error
traducido */ })
```

Cómo lo explicas: *«El thunk lee el `userId` de la sesión con `getState`, llama a la API y devuelve el resultado; si falla usa `rejectWithValue` para entregar un error **ya traducido** (`SymptomsError`) al reducer `rejected`. Los tres reducers mantienen el estado de la pantalla: cargando, listo o error.»*

8.6 Cuatro patrones avanzados que el repo usa (¡esto distingue un 100 %!)

1. **`condition` para no lanzar un thunk sin sesión** — `fetchAreaProgress` en `activities-slice.ts`:

   ```
   { condition: (_, { getState }) => Boolean(getState().auth.session) }
   ```

   Se descubrió un **bug real**: al cerrar sesión, el store se reinicia con la pestaña de Actividades aún montada; el efecto volvía a lanzar la carga **sin sesión** y dejaba un error fijo («No hay sesión activa») para la cuenta siguiente. Con `condition`, si no hay sesión el thunk **ni siquiera se ejecuta** (no genera `pending/rejected`).
2. **`condition` para no repetir un efecto de un solo uso** — `openAuthLink` en `auth-slice.ts`: `condition: (url, { getState }) => getState().auth.linkFlow.handledUrl !== url`. El `code` de un enlace de correo **es de un solo uso**, así que no se canjea dos veces; si se cancela por `condition`, el reducer `rejected` detecta `action.meta.condition` y **no toca el estado**.
3. **`requestId` para descartar respuestas viejas** — `doctor-slice.ts`, `fetchPatientSummary`:

   ```
   .addCase(fetchPatientSummary.pending, (state, action) => { state.summary = null;
   state.summaryRequestId = action.meta.requestId; })
   .addCase(fetchPatientSummary.fulfilled, (state, action) => {
     if (state.summaryRequestId !== action.meta.requestId) return;   // llegó tarde de una
   petición anterior
     state.summary = action.payload;
   })
   ```

Por qué: si el médico cambia de «Mes» a «3 meses» rápido, la respuesta lenta del período anterior **no debe pisar** a la nueva (daría cifras de un rango bajo la etiqueta de otro). Es una **condición de carrera** resuelta con el `requestId` único que genera cada thunk.

4. **Estado local vs. global:** el **borrador del asistente** y la **sesión de actividad** viven en Redux (se comparten entre pantallas y sobreviven a la navegación); los detalles puramente visuales (p. ej. si el campo de contraseña está oculto: `useState` en `TextField`) se quedan **locales**.

8.7 La máquina de estados de las actividades (`activities-slice.ts`)

`session.phase` puede ser: `'memorizar' → 'responder' → 'resultado' → 'completada'`. Los reducers **validan la fase** antes de actuar (p. ej. `memoriaShapeTapped` retorna sin hacer nada si `phase !== 'responder'`). Esto evita estados imposibles (tocar una figura cuando ya se mostró el resultado).

- `sessionStarted` arranca (fase inicial `'memorizar'` **solo** si es Memoria; las demás empiezan en `'responder'` — **bug real**: antes todas empezaban en `'memorizar'` y Atención salía en blanco porque su bloque de render dependía de `memoriaRound`).
- `roundContinued` avanza de ronda; en la **última ronda** calcula `newLevel = nextLevel(levelAtStart, MAX_LEVEL[area], roundsCorrect)` y pasa a `'completada'`.
- `memoriaShapeTapped` **falla rápido**: un toque fuera de orden termina la ronda de inmediato.
- `movimientoSessionFinished` siempre sube de nivel (participación, no precisión): `newLevel = Math.min(levelAtStart + 1, MAX_LEVEL.movimiento)`.

8.8 Por qué Redux Toolkit y no Context (la defensa de D-005)

- **Context** sirve para valores simples que cambian poco (tema, idioma). **No trae** manejo de asincronía, ni herramientas de trazabilidad, y **cualquier cambio re-renderiza a todos los consumidores**.
- Aquí hay **estado compartido y asíncrono**: sesión, vínculos y permisos, borradores entre pantallas, caché por mes, reseteo global al cerrar sesión (`resetAll`).
- RTK reduce el *boilerplate* clásico de Redux (`createSlice`, `createAsyncThunk`, Immer) y da **acciones nombradas y trazables** (`symptoms/fetchMonthLogs/pending`).
- **Costo reconocido (D-005):** «algo de boilerplate que todos deben entender» — por eso esta sección.

8.9 Preguntas típicas

- *«¿Qué pasa si dos pantallas necesitan el mismo dato?»* Ambas lo leen con `useAppSelector` del mismo slice; hay una sola fuente de verdad.
- *«¿Por qué `rejectWithValue`?»* Para que el `payload` del `rejected` sea **mi error tipado y traducido**, no el `Error` genérico serializado de Redux.
- *«¿Qué es `action.meta.arg`?»* El argumento con el que se llamó al thunk; se usa p. ej. en `register.fulfilled` para guardar `pendingConfirmationEmail = action.meta.arg.email`.
- *«¿Por qué `getState` dentro del thunk?»* Para leer el `userId` de la sesión sin que la pantalla tenga que pasarlo.
- *«¿Dónde se limpian los datos al cerrar sesión?»* `resetAll` (`src/store/actions.ts`) + reducer raíz (`store.ts`) + `onAuthStateChange` en `_layout.tsx`.

---

9. Capa de datos, sesión segura y manejo de errores

9.1 El cliente de Supabase (`src/lib/supabase.ts`)

```
import "react-native-get-random-values";
import "react-native-url-polyfill/auto";
…
const supabaseUrl = process.env.EXPO_PUBLIC_SUPABASE_URL;
const supabasePublishableKey = process.env.EXPO_PUBLIC_SUPABASE_PUBLISHABLE_KEY;
if (!supabaseUrl || !supabasePublishableKey) throw new Error("Faltan EXPO_PUBLIC_SUPABASE_URL …");

export const supabase = createClient(supabaseUrl, supabasePublishableKey, {
  auth: { storage: new LargeSecureStore(), autoRefreshToken: true, persistSession: true,
          detectSessionInUrl: false, flowType: 'pkce' },
});
```

- **Los dos `import` de polyfill** (`get-random-values`, `url-polyfill`) existen porque React Native **no trae** `crypto.getRandomValues` ni `URL` completos, y Supabase los necesita.
- **Variables de entorno:** el prefijo **`EXPO_PUBLIC_`** hace que Expo las incluya en el bundle. **La clave `publishable` es pública por diseño** (equivale a la antigua `anon key`); la seguridad **no depende de esconderla** sino de **RLS**. **Nunca** se usa la `service_role` en la app. `.env` está en `.gitignore` y **nunca se commiteó** (verificado: `git log -- .env` no devuelve nada; solo se versiona `.env.example`).
- `persistSession: true` + `autoRefreshToken: true`: la sesión sobrevive a reinicios y el token se renueva solo.
- `detectSessionInUrl: false`: en móvil no hay URL de navegador que leer.
- `flowType: 'pkce'`: flujo OAuth más seguro para apps (código de un solo uso + verificador guardado en el dispositivo).

9.2 `LargeSecureStore`: el problema y la solución (¡pregunta probable!)

El problema (bug real encontrado en T0.5): `expo-secure-store` en Android permite **~2048 bytes por valor**, pero el **JWT de sesión de Supabase es más grande**. Con el adaptador simple, `setItemAsync` fallaba **en silencio** y la sesión **nunca sobrevivía a un reinicio** (el usuario parecía logueado, pero al reabrir se perdía).

La solución (patrón recomendado por Supabase):

```
class LargeSecureStore implements SupportedStorage {
  private async encrypt(key, value) {
    const encryptionKey = crypto.getRandomValues(new Uint8Array(256 / 8));         // clave AES-256
aleatoria
    const cipher = new aesjs.ModeOfOperation.ctr(encryptionKey, new aesjs.Counter(1));
    const encryptedBytes = cipher.encrypt(aesjs.utils.utf8.toBytes(value));
    await SecureStore.setItemAsync(key, aesjs.utils.hex.fromBytes(encryptionKey)); // la CLAVE
(corta) va a SecureStore
    return aesjs.utils.hex.fromBytes(encryptedBytes);                               // el blob
cifrado (largo)
  }
  …
  async getItem(key) { const encrypted = await AsyncStorage.getItem(key); if (!encrypted) return
null; return this.decrypt(key, encrypted); }
  async setItem(key, value) { const encrypted = await this.encrypt(key, value); await
AsyncStorage.setItem(key, encrypted); }
  async removeItem(key) { await AsyncStorage.removeItem(key); await
```

```
SecureStore.deleteItemAsync(key); }
}
```

Cómo lo explicas (en orden):

1. La sesión se **cifra con AES-256 en modo CTR** (`aes-js`) con una **clave aleatoria** generada con `crypto.getRandomValues`.
2. La **clave de cifrado** (32 bytes → 64 caracteres hex, **cabe** en el límite) se guarda en **SecureStore** (que usa el Keystore de Android).
3. El **texto cifrado** (grande) se guarda en **AsyncStorage**.
4. Para **leer**: se toma el blob de AsyncStorage, la clave de SecureStore, y se descifra.
5. `implements SupportedStorage`: cumple la interfaz que Supabase espera (`getItem/setItem/removeItem`).

Por qué no guardar solo en SecureStore: por el límite de ~2 KB. **Por qué no solo en AsyncStorage:** no está cifrado; cualquiera con acceso al almacenamiento leería el token.

> **Nota honesta:** CTR con `Counter(1)` fijo y clave regenerada en cada `setItem` es el patrón estándar documentado; como la clave cambia **en cada escritura**, no se reutiliza el par clave/contador.

9.3 El patrón `api.ts` y la traducción de errores

Cada módulo tiene un `api.ts` con **funciones que llaman a Supabase y lanzan errores ya traducidos**. Ejemplo (`features/symptoms/api.ts`):

```
function toSymptomsError(error: unknown): SymptomsError {
  if (isPostgrestLikeError(error)) {
    if (error.code === 'PGRST205') return { errorClass: 'servidor', message: 'Falta crear la tabla…'
};
    if (error.code === 'PGRST301' || error.code === '401') return { errorClass: 'validacion',
message: 'Tu sesión expiró…' };
    if (error.code.startsWith('23')) return { errorClass: 'validacion', message: 'Revisa los datos
del registro' };
    return { errorClass: 'servidor', message: 'Falló el servidor, inténtalo de nuevo en un momento'
};
  }
  if (error instanceof TypeError) return { errorClass: 'sin-conexion', message: 'Revisa tu conexión
e inténtalo de nuevo' };
  console.warn('[symptoms] error inesperado', error);
  return { errorClass: 'inesperado', message: 'Algo salió mal, inténtalo de nuevo' };
}
```

Clases de error: `sin-conexion` · `servidor` · `validacion` · `codigo-invalido` (solo cuidador/médico) · `inesperado`. En auth: `credenciales-invalidas`, `correo-sin-confirmar`, etc.

Dos decisiones que debes poder justificar:

1. **Duck-typing en lugar de `instanceof PostgrestError`:** se descubrió un **bug real** — `error instanceof PostgrestError` **nunca era verdadero** en este runtime, así que cualquier error de Supabase caía en el mensaje genérico «Algo salió mal». Se cambió a **detección por forma** (`'code' in error && 'message' in error`).
2. **`TypeError` = sin conexión:** cuando `fetch` falla por red, lanza un `TypeError`; así se muestra «Revisa tu conexión» (manejo de estado **offline** exigido por el enunciado).

Códigos que conviene reconocer: `PGRST205` = tabla no existe en el caché de esquema (falta una migración); `PGRST202/42883` = función no existe; `PGRST301/401` = sesión expirada; códigos `23xxx` = violación

de restricción (check, unique, not null).

9.4 Cómo la pantalla usa los estados (`ScreenState`)

`src/components/screen-state.tsx` muestra **carga, error o vacío** con accesibilidad incluida:

```
<View accessible
      accessibilityRole={kind === 'error' ? 'alert' : undefined}
      accessibilityState={{ busy: kind === 'cargando' }}
      accessibilityLiveRegion={kind === 'error' ? 'assertive' : 'polite'}>
```

- `accessibilityLiveRegion` hace que TalkBack **anuncie** cuando aparece (cargando = `polite`, error = `assertive`).
- `accessibilityState.busy` avisa que algo está cargando.
- Lo usan 13 pantallas (calendario, actividades, progreso, el módulo del cuidador, el del médico, Mi médico, Permisos, etc.): consistencia (RNF-008).

9.5 Offline: qué hace y qué no

- **Qué hace:** detecta el fallo de red (`TypeError`) y muestra «Revisa tu conexión e inténtalo de nuevo»; en actividades, si falla guardar el nivel **no bloquea** (el nivel ya está en la sesión local; `persistAreaLevel.rejected` solo marca `saveStatus = 'error'`).
- **Qué NO hace (limitación declarada): no hay caché offline ni cola de sincronización**; sin conexión no se pueden ver ni guardar datos nuevos. Está en el *roadmap* v1.2.

---

10. Backend y base de datos: Supabase, Postgres, RLS, funciones y triggers

> **Preguntas literales probables:** «¿Qué backend usan?», «¿cómo consumen la API?», «¿cómo protegen los datos del paciente?», «¿qué es RLS?», «¿por qué no validan permisos en la app?». Esta sección las responde.

10.1 Qué es Supabase y qué parte usamos

Supabase es un *backend como servicio* sobre **PostgreSQL**. Usamos tres piezas:

1. **Auth:** cuentas, sesiones (JWT), recuperación de contraseña, correo de confirmación.
2. **Postgres + API automática (PostgREST):** cada tabla se expone como una API REST. El cliente `supabase-js` la usa con `supabase.from('tabla').select/insert/update/delete`.
3. **RPC (funciones):** funciones SQL que se llaman desde el cliente con `supabase.rpc('nombre', { parámetros })`.

No hay un servidor propio (Node/Express): el «backend» es **la base de datos con sus reglas**. Por eso la seguridad está en Postgres (RLS, funciones y triggers), no en código de aplicación. Esto es coherente con RT-004: es un backend de **cuentas y datos**, **no** un backend clínico ni de diagnóstico.

Cómo se «consume la API» en el código: nunca con `fetch` a mano. Cada módulo tiene `features/X/api.ts` con funciones como:

```
const { data, error } = await supabase
  .from('symptom_logs')
```

```
  .select('id, symptom_type, severity, note, occurred_at')
  .eq('patient_id', patientId)
  .gte('occurred_at', start)
  .lt('occurred_at', end)
  .order('occurred_at', { ascending: true });
if (error) throw toSymptomsError(error);
```

Se lee: «de la tabla `symptom_logs`, seleccionar estas columnas, donde `patient_id` es igual al usuario, y la fecha esté en el rango `[start, end)`, ordenado ascendente». **Intervalo semiabierto** `[start, end)`: `gte` incluye el inicio y `lt` excluye el fin, para que un instante nunca caiga en dos meses.

10.2 El modelo de datos (qué tabla guarda qué)

Todas las tablas referencian `auth.users(id)` con **`on delete cascade`**: al borrar la cuenta, **todo** lo del usuario se borra (derecho de supresión; exigido también por las tiendas de apps).

Tabla	Columnas clave	Para qué
`symptom_logs`	`patient_id`, `symptom_type` (fatiga, vision_borrosa, dolor, debilidad, equilibrio, otro), `severity` (leve/moderado/fuerte), `note`, `occurred_at`	Registro de síntomas (RF-001/002)
`activity_progress`	`patient_id`, `area`, `level` (único por `patient_id,area`)	Nivel actual por área
`activity_sessions`	`patient_id`, `area`, `completed_at`	Historial de sesiones completadas (para Progreso)
`caregiver_links`	`patient_id`, `caregiver_id` (null hasta reclamar), `invite_code`, `status`, `share_sintomas/progreso/recordatorios`, `patient_name`	Vínculo paciente–cuidador y permisos
`caregiver_observations`	`link_id`, `caregiver_id`, `patient_id`, `category`, `note`	Bitácora **privada** del cuidador (el paciente **no** la ve)
`caregiver_cheers`	`link_id`, `message`, `read_at`	Mensajes de ánimo cuidador → paciente
`reminders`	`patient_id`, `created_by`, `category` (medicación/cita/bienestar), `scheduled_at`, estado	Recordatorios
`doctor_links`	como `caregiver_links` + `share_notas` (apagado), `expires_at`, `last_viewed_at`, `invite_expires_at`, `professional_id`	Vínculo paciente–médico
`claim_attempts`	`user_id`, `attempted_at`	Intentos **fallidos** de reclamar un código (para el límite)

`CHECK` constraints: los valores permitidos están **también en la base** (`check (severity in ('leve','moderado','fuerte'))`), no solo en la app. **Índices:** p. ej. `symptom_logs_patient_occurred_idx on (patient_id, occurred_at desc)` acelera «los registros de este paciente por fecha».

10.3 Row Level Security (RLS): la idea central

Definición: *RLS es un mecanismo de Postgres que filtra, **fila por fila**, qué puede ver o modificar cada usuario, usando políticas.* Se activa por tabla y **sin políticas nadie accede a nada**:

```
alter table public.symptom_logs enable row level security;

create policy "symptom_logs_select_own" on public.symptom_logs
```

```
  for select using (auth.uid() = patient_id);

create policy "symptom_logs_insert_own" on public.symptom_logs
  for insert with check (auth.uid() = patient_id);
```

- `auth.uid()` devuelve el **id del usuario autenticado** (sale del JWT que manda la app; **no se puede falsificar** desde el cliente).
- `using (...)` = qué filas **existentes** puede ver/tocar (para `select`, `update`, `delete`).
- `with check (...)` = qué valores puede **escribir** (para `insert` y para el resultado de un `update`).
- Las políticas de varios tipos **se suman con OR** (si alguna permite, se permite). Por eso el acceso del cuidador a `activity_progress` es **otra política adicional** que se **suma** a «select own» sin reemplazarla.

Por qué RLS y no validar en la app (la respuesta clave): la clave del cliente **es pública**; cualquiera podría llamar a la API directamente saltándose la app. Si la regla vive en la app, se evita; si vive en la base, **no hay forma de saltarla**.

10.4 Quién puede qué (la tabla que debes saber)

Recurso	Paciente	Cuidador (vínculo aceptado)	Médico (aceptado y vigente)
`symptom_logs`	Todo lo suyo	Vía función `shared_symptom_logs`, **sin notas**, si `share_sintomas`	Vía `shared_symptom_logs`; **notas solo si** `share_notas`
`activity_progress`, `activity_sessions`	Todo lo suyo	**Lectura** si `share_progreso`	**Lectura** si `share_progreso` (y no caducado)
`reminders`	Todo lo suyo	**Crear** y **marcar «hecho»** si `share_recordatorios` (no editar ni borrar)	—
`caregiver_observations`	**No las ve (a propósito)**	Las suyas	—
`caregiver_cheers`	Las lee y las descarta	Las envía	—
Vínculos	Crear, aceptar, rechazar, revocar	Reclamar código con `claim_invite`	Reclamar código con `claim_doctor_invite`; **no modifica** el vínculo

10.5 Funciones `security definer`: por qué y cómo

Problema: RLS filtra **filas**, **no columnas**, y no puede «ver» datos de otra persona sin un permiso general. Ejemplos: ocultar la columna `note` al cuidador/médico según un permiso; o dejar que un cuidador **reclame un código** que aún no le pertenece (no puede leer esa fila por RLS).

Solución: una función `security definer` se ejecuta con los **privilegios de su dueño** (no del que llama) y **contiene la regla**. Se cierra el acceso directo y solo se permite pasar por la función:

```
create or replace function public.claim_invite(p_code text) returns setof public.caregiver_links
language plpgsql security definer set search_path = public, auth as $$ … $$;

revoke all on function public.claim_invite(text) from public, anon;
grant execute on function public.claim_invite(text) to authenticated;
```

- `set search_path = public, auth`: fija el esquema de búsqueda para evitar ataques de suplantación de objetos (buena práctica con `security definer`).
- `revoke … from public, anon` + `grant … to authenticated`: solo usuarios con sesión pueden ejecutarla.

Las funciones del proyecto:

Función	Qué hace	Migración
`claim_invite(p_code)`	El cuidador reclama un código: marca `caregiver_id`, `caregiver_email/name` (tomados de **su sesión**, no del cliente) y `status='solicitado'`; **cuenta intentos fallidos** y bloquea a los 10 en 15 min (`P0003 rate_limited`)	`20261114`
`delete_my_account()`	Borra al usuario de `auth.users`; el `cascade` borra todo lo demás	`20261114`
`create_doctor_invite()`	Genera el **código en el servidor** (8 caracteres, $32^8 \approx 1{,}1{\cdot}10^{12}$ combinaciones, alfabeto sin 0/O/1/I, 7 días de vigencia) y devuelve la invitación vigente si ya existe	`20261115`
`claim_doctor_invite(p_code, p_professional_id)`	Igual que `claim_invite` para médicos, pero con un **candado por usuario** (`pg_advisory_xact_lock`) que hace **atómico** el conteo de intentos	`20261115`
`mark_doctor_viewed(p_link)`	Guarda `last_viewed_at` (el paciente ve «último acceso»)	`20261115`
`shared_symptom_logs(p_patient, p_from, p_to)`	**Única vía** para que un tercero lea síntomas: valida vínculo, permisos y vigencia; **oculta `note`** salvo médico con `share_notas`; limita a **2 años** (minimización de datos); si no hay acceso devuelve **cero filas** (no error)	`20261115`

Cómo se llaman desde la app: `supabase.rpc('shared_symptom_logs', { p_patient: patientId, p_from: fromISO, p_to: toISO })` (`features/symptoms/api.ts`, `listSharedSymptomLogs`) y `supabase.rpc('delete_my_account')` (`features/auth/api.ts`).

`shared_symptom_logs` explicada (¡buen candidato a pregunta!): primero un `with access as (…)` calcula dos cosas: `caregiver_ok` (¿hay un vínculo de cuidador aceptado con `share_sintomas`?) y `doctor_notes` (`bool_and(dl.share_notas)` de los vínculos de médico aceptados, con `share_sintomas`, **no caducados**; es `null` si no hay acceso, `true/false` si lo hay). Luego el `select` devuelve filas solo si `caregiver_ok` **o** `doctor_notes is not null`, y la nota se entrega **solo** cuando `doctor_notes is true`: `case when a.doctor_notes is true then s.note else null end`. Con varios vínculos de médico manda el **más restrictivo** (`bool_and`).

10.6 Triggers: proteger columnas y completar datos

Un **trigger** es una función que Postgres ejecuta **automáticamente** antes/después de un `insert/update/delete`.

Trigger	Qué evita / hace
`reminders_guard_caregiver_update`	RLS no restringe columnas: sin esto un cuidador podría **cambiar título/hora** de un recordatorio. El trigger rechaza (`caregivers_can_only_mark_done`) cualquier cambio salvo el estado por parte de quien no es el paciente
`doctor_links_guard_patient_update`	El paciente solo puede cambiar `status` (con **transiciones válidas**), `share_*`, `expires_at`, `responded_at`; **no** `doctor_id`, nombre/correo del médico, código, etc. Si no, un atacante podría fabricar un «médico» falso. Al pasar a `aceptado` copia el `patient_name` **desde la cuenta**, no desde el cliente
`caregiver_links_set_patient_name`	Llena `patient_name` **solo cuando `status='aceptado'`** (el paciente ya consintió) y **sobrescribe** cualquier valor del cliente (D-019)

10.7 La máquina de estados de un vínculo (RF-012, el consentimiento)

```
pendiente  ──(el otro perfil reclama con claim_*_invite)──►  solicitado
solicitado ──(SOLO el paciente acepta, eligiendo permisos)──►  aceptado
solicitado ──(el paciente rechaza)──────────────────────────►  rechazado
aceptado   ──(el paciente retira el acceso)─────────────────►  revocado
```

- **El paciente es el único que puede pasar a `aceptado`** (política `…_update_as_patient`): eso **es** el consentimiento explícito.
- Al aceptar elige 3 permisos: **Síntomas, Progreso, Recordatorios** (cuidador) o **Síntomas, Notas, Progreso + duración** (médico).
- **Revocar corta el acceso de inmediato**: las políticas y funciones comprueban `status = 'aceptado'` **en cada consulta**.
- **Caducidad del médico:** `expires_at` (3 meses, 6 meses, 1 año o sin fecha). Las políticas y `shared_symptom_logs` incluyen `(expires_at is null or expires_at > now())`: **lo comprueba la base de datos**, no la app.

10.8 Código de invitación: cómo se genera y reclama

Cuidador (`features/caregiver/api.ts`):

```
const CODE_ALPHABET = 'ABCDEFGHJKLMNPQRSTUVWXYZ23456789'; // sin 0/O ni 1/I
export function generateInviteCode(): string {
  const bytes = new Uint8Array(6);
  crypto.getRandomValues(bytes);
  let code = '';
  for (let i = 0; i < bytes.length; i++) code += CODE_ALPHABET[bytes[i] % CODE_ALPHABET.length];
  return code;
}
```

- **6 caracteres de un alfabeto de 32** (sin `0/O/1/I` porque se confunden al transcribir). `256` es múltiplo de `32`, por lo que `byte % 32` **no introduce sesgo**.
- `getOrCreatePatientInvite`: el paciente tiene como máximo **una invitación pendiente** a la vez; si ya hay, la reutiliza.
- **Limitación declarada (¡dila tú!):** en el cuidador **el código lo genera el cliente** y se inserta con una política `insert` del paciente. El índice único parcial (`caregiver_links_pending_code_uniq … where status = 'pendiente'`) responde «ya existe» ante un duplicado, lo que da una pista (**oráculo**) para adivinar códigos. Y el límite de intentos de `claim_invite` **no es atómico** (una ráfaga simultánea puede pasar el límite). **En el médico ya se arregló** (código en servidor + candado). Para el cuidador está como mejora **v1.1**.

Médico: el código lo genera el servidor (`create_doctor_invite`), no hay política de `insert` para el cliente.

10.9 Auditoría de seguridad: qué se encontró y se corrigió

La auditoría del 2026-11-10 (`supabase/migrations/20261114…`) y la del perfil médico (2026-11-11) encontraron:

Hallazgo	Riesgo	Corrección
`select_unclaimed_by_code` no comparaba el código	**Cualquiera** (incluso sin sesión) podía **listar todos los códigos pendientes** y los `patient_id`	Se **eliminó** esa política; se reclama **solo** con `claim_invite`

Hallazgo	Riesgo	Corrección
La política de reclamo dejaba al cuidador reescribir `patient_id`, permisos y nombre	Podía auto-concederse acceso a otro paciente	La función fija esos campos **en el servidor**
`reminders`: cuidador podía editar cualquier columna	Manipular recordatorios del paciente	Trigger `reminders_guard_caregiver_update`
`caregiver_observations.insert` solo miraba `link_id`	Insertar con un `patient_id` distinto al del vínculo	Política corregida (exige `cl.patient_id = caregiver_observations.patient_id`)
Textos libres sin límite	Abuso/almacenamiento	`check (char_length(...) between 1 and N)` (1000 / 280 / 120)
Borrado de cuenta ausente	Requisito de tiendas y de protección de datos	`delete_my_account()` + `on delete cascade`
Médico: código elegido por el cliente	Oráculo de códigos	`create_doctor_invite` en servidor
Médico: límite de intentos no atómico	Ráfagas superaban el límite	`pg_advisory_xact_lock` por usuario
Médico: paciente podía escribir `doctor_id`, nombre, etc.	«Médico falso»	Trigger `doctor_links_guard_patient_update`
Bug: `share_sintomas` del cuidador no tenía efecto	El cuidador **nunca** podía leer síntomas	`shared_symptom_logs` lo aplica

10.10 Migraciones

Son **10 archivos SQL** con marca de tiempo (`2026110800…` a `2026111700…`). **No hay CLI de Supabase enlazada**: se ejecutan **a mano, en orden, en el SQL Editor** (README lo lista). Son **idempotentes** (`create table if not exists`, `drop policy if exists`, `create or replace function`): se pueden correr varias veces. Si se omite una, la app lo detecta (`PGRST205`/`PGRST202`) y muestra «Falta crear la tabla… (ver supabase/migrations)». **Seed:** `supabase/seed/demo_data_completo.sql` carga ~5 meses de datos de demo (con placeholders `@EJEMPLO.COM`, sin correos reales).

10.11 Preguntas típicas

- *«¿Qué pasa si alguien intercepta la clave publishable?»* Nada grave: por sí sola solo permite lo que RLS permita a un usuario **autenticado** sobre **sus** datos; sin sesión válida no ve filas.
- *«¿Cómo evitan que el cuidador lea las notas del síntoma?»* La función `shared_symptom_logs` devuelve `note = null` para cuidadores; y el cuidador **no tiene política de `select` directo** sobre `symptom_logs`.
- *«¿Y si el paciente revoca mientras el cuidador tiene la pantalla abierta?»* La siguiente consulta ya no devuelve filas (se evalúa `status = 'aceptado'` en cada consulta); el acceso se corta en el servidor.
- *«¿Por qué una tabla `doctor_links` aparte y no un rol en `caregiver_links`?»* (D-016.3) Las políticas del cuidador solo exigen `status='aceptado'` y le darían al médico **poderes de escritura** (recordatorios, mensajes, observaciones).
- *«¿Por qué el nombre del paciente se copia en la fila?»* El cuidador **no puede leer** `auth.users` (RLS); una función/trigger copia el nombre cuando hay consentimiento.

10.12 ¿Por qué no hay un backend propio? (la pregunta trampa)

> **Pregunta muy probable:** *«Supabase es la base de datos; ¿no debería haber un backend intermediario con la lógica de negocio? ¿Así no queda expuesta la base?»* **Idea central:** no es que «no haya backend»; **el backend es Supabase** y la seguridad se aplica **en el servidor**, no en la app. Lo que no hay es un servidor **escrito por nosotros** (Node/Express, FastAPI…).

a) Supabase no es «solo la base de datos»

Es Postgres **más** una capa de servidor incluida:

- **Auth:** emite y valida los **JWT**, guarda las contraseñas con *hash*.
- **PostgREST:** la **API REST** que recibe las peticiones de la app. Antes de tocar una tabla, Postgres aplica **RLS** con el usuario del JWT.
- **Funciones SQL (RPC) y triggers:** código que se ejecuta **en el servidor**, con lógica de negocio.

La app nunca se conecta directo a Postgres (con usuario y contraseña de la base): pasa por esa API, que exige un token válido y aplica las políticas **en cada consulta**. Es el patrón **«BaaS + RLS»** (*backend as a service*), usado y recomendado por Supabase.

b) Por qué la base no queda vulnerable sin intermediario

1. La clave del cliente (`publishable`) **es pública por diseño**: por sí sola solo permite lo que RLS autorice a un usuario autenticado sobre **sus** datos.
2. Aunque alguien modifique la app o llame a la API a mano (Postman, `curl`), **RLS se evalúa igual**: la regla vive en la base, no en la pantalla.
3. Las reglas críticas están en el servidor: «solo el paciente acepta un vínculo» (política), «el cuidador no ve las notas» (`shared_symptom_logs`), «el cuidador solo marca recordatorios como hechos» (trigger), límite de intentos de códigos (`claim_invite`), borrado de cuenta (`delete_my_account`).
4. **La clave secreta (`service_role`) nunca está en la app.** Es la única que se salta RLS.

c) Las capas clásicas sí existen; cambia quién las implementa

Concepto de backend clásico	Qué hace	Dónde está en Neurotrack EM
API REST	Endpoints HTTP	**PostgREST** la genera desde las tablas (`/rest/v1/symptom_logs`); no se escribe a mano
Router (de backend)	URL → controlador	Supabase: cada tabla es un recurso; cada función es `/rest/v1/rpc/<nombre>`
Middleware de autenticación	Validar el token	Supabase valida el JWT; de ahí sale `auth.uid()`
Controller	Recibir, validar, responder	**Funciones RPC** (`claim_invite`, `shared_symptom_logs`, `delete_my_account`…)
Capa de servicio (negocio)	Reglas del dominio	Reglas **sensibles** en SQL (funciones, triggers); **no sensibles** en `src/features/*/logic.ts`
Autorización	¿Puede hacer esto?	**RLS**
Validación	Datos correctos	`CHECK` constraints, límites de largo, triggers; formularios en la app

Concepto de backend clásico	Qué hace	Dónde está en Neurotrack EM
Repositorio / DAO	Encapsular consultas	`src/features/*/api.ts` (único que habla con Supabase)
Modelos / DTO	Forma de los datos	Tablas SQL + `types.ts`; `api.ts` mapea `snake_case` → `camelCase` (`toSymptomLog`)
Manejo de errores	Respuestas de error	`toSymptomsError`, `toAuthError`… con clases (`sin-conexion`, `servidor`, `validacion`)
Migraciones	Versionar el esquema	`supabase/migrations/` (10 archivos)

> **Ojo con «router»:** en el proyecto **sí hay un router, pero de frontend**: **Expo Router** decide qué *pantalla* mostrar (`src/app/`). No es un router de API. Si el profesor pregunta por «el router», aclara a cuál se refiere.

d) Cuándo SÍ convendría un backend propio (y nuestras debilidades reales)

Situación que justifica un intermediario	Nuestro caso
Secretos de terceros (claves de otras APIs) que no pueden ir en el móvil	No usamos ninguno
Reglas de negocio complejas o que no se pueden falsear	Las sensibles están en SQL. **Pero** la subida de nivel (`nextLevel`) la calcula la **app**: un usuario podría falsear **su propio** nivel. Impacto bajo (no hay puntaje competitivo ni dato clínico)
Validación centralizada	Solo `CHECK`, longitudes y triggers; no un validador de negocio completo
Auditoría de accesos a datos	Solo `last_viewed_at` del médico
Tiempo real, notificaciones push, tareas programadas	No los necesitamos (los recordatorios son dentro de la app)
Límites de uso	Solo para reclamar códigos

Debilidades que se declaran:

- El **código de invitación del cuidador** lo genera el cliente y su límite de intentos no es atómico. En el médico ya se movió al servidor.
- El **consentimiento** (`accepted_terms_at`) lo envía el cliente: constancia razonable, no prueba infalible.
- **RLS es fácil de configurar mal**, y nos pasó: la auditoría encontró que se podían listar códigos de invitación y reescribir columnas (sección 10.9). Está corregido, pero ese es el riesgo del enfoque: **un error en una política expone datos**, mientras que un intermediario añade **una segunda barrera**.

e) Opinión objetiva: ¿era buena práctica tener backend aquí?

- **Para este alcance (semestre, 3 personas, capa gratuita), no hacía falta.** La lógica principal es «quién puede ver qué», que RLS resuelve en la base; no hay secretos ni tiempo real. Un servidor propio habría añadido despliegue, mantenimiento y **más superficie de ataque** sin mejorar la seguridad de lo que hoy hace la app.
- **Lo que sí sería buena práctica** es mover unas pocas piezas más al servidor: código de invitación del cuidador, validación de niveles, constancia de consentimiento y auditoría. **No exige**

un servidor aparte: bastan funciones SQL (como ya hicimos) o **Supabase Edge Functions** (funciones de servidor dentro de la misma plataforma).
- **Si pasara a producción** con datos de salud reales, conviene una capa de servidor explícita: auditoría, validación centralizada, monitoreo y revisión legal (Ley 1581).

f) Contraste con un proyecto que sí lo necesita: EA-FC-PROYECTO

El repositorio `SamuelBlanco-65/EA-FC-PROYECTO` (torneo de EA FC) **sí tiene backend propio**: **FastAPI en Python**, desplegado en **Render**, con todas las capas clásicas: `app/main.py` (entrada y *middleware*), `app/api/` (routers/controllers: `GET /matches/{id}`, `POST /matches/{id}/events`…), `app/services/` (negocio), `app/domain/` (máquina de estados del partido, *round robin*), `app/repositories/`, `app/schemas/` (Pydantic), WebSocket propio (`app/realtime/`) y pruebas de integración. La app móvil **no toca la base**: llama a esa API con `Authorization: Bearer <token>`. La base sigue siendo Supabase, pero detrás del backend, que usa la **clave secreta** (se salta RLS y por eso el servicio autoriza cada petición) o actúa **como el usuario** (con RLS).

	EA-FC-PROYECTO	**Neurotrack EM**
Arquitectura	App → **API propia (FastAPI)** → Supabase	App → **API de Supabase** (PostgREST)
Lógica de negocio	Servicios en Python	Funciones SQL y triggers + `logic.ts`
Autorización	El backend valida (y puede usar `service_role`)	**RLS** en cada consulta
Secretos en servidor	Sí (clave secreta en Render)	No hay; la app solo tiene la clave pública
Tiempo real	WebSocket propio	No lo necesita
Costo operativo	Desplegar y mantener un servidor (en Render gratuito se duerme a los 15 min; arranque en frío ~1 min)	Nada extra que desplegar

Por qué allí sí: reglas de **competencia** (quién registra un gol; **el servidor calcula el marcador** para que nadie haga trampa), **tiempo real** y **clave secreta** que no puede ir en el móvil. **Aquí no:** la lógica es de permisos sobre datos personales (RLS) y no hay secretos ni tiempo real. **El argumento: la arquitectura se eligió según el problema.**

g) Respuesta modelo para decir en voz alta (~45 s)

> «Usamos Supabase como backend como servicio. La app no accede directo a la base: pasa por la API REST de Supabase, que valida el JWT y aplica Row Level Security en cada consulta, así que aunque alguien se salte la app no puede leer datos ajenos. La lógica de negocio sensible —permisos, consentimiento, ocultar notas, límites de intentos, borrado de cuenta— se ejecuta **en el servidor**, en funciones SQL y triggers. Las capas de un backend clásico existen igual: la API la genera PostgREST, la autorización es RLS, las funciones hacen de controladores y en la app `api.ts` funciona como repositorio. Para el alcance del proyecto un servidor propio habría sido sobreingeniería, porque no manejamos secretos ni tiempo real. Reconocemos el trade-off: un intermediario añade una segunda barrera y permitiría, por ejemplo, validar en servidor la subida de nivel o generar el código del cuidador. Si fuera a producción, moveríamos esas reglas a Edge Functions.»

---

11. Autenticación, correos y consentimiento

11.1 Los estados de `auth` (`auth-slice.ts`)

`status`: **`bootstrapping`** (arranque: ¿hay sesión guardada?) → **`unauthenticated`** / **`authenticating`** (login en curso) → **`authenticated`**. El `error` es un `AuthError` tipado (`authClass`: `sin-conexion`, `credenciales-invalidas`, `correo-sin-confirmar`, `validacion`, `servidor`, `inesperado`). `pendingConfirmationEmail` guarda el correo cuando el **registro fue correcto pero falta confirmar** (no se autentica en falso).

11.2 El bug de la «sesión falsa» (T0.5) — pregunta muy probable

En `signIn`/`signUp` se armaba la sesión a partir de `data.user`, no de `data.session`. Con **confirmación de correo obligatoria**, un registro sin confirmar **parecía autenticar** (había `user`) y redirigía a Inicio, pero **no había sesión real**: al reiniciar se perdía. **Arreglo** (`features/auth/api.ts`): `toSession(data.session?.user)` — **si no hay `session`, no hay sesión**. En `signUp` devuelve `null` en ese caso y el reducer `register.fulfilled` pone `pendingConfirmationEmail`.

11.3 `signUp`: detalles que muestran dominio

```
const { data, error } = await supabase.auth.signUp({
  email, password,
  options: { data: { full_name: name, ...buildConsentMetadata() }, emailRedirectTo:
Linking.createURL(CONFIRM_REDIRECT_PATH) },
});
…
if (data.user && data.user.identities?.length === 0) throw { authClass: 'validacion', message: 'Ya
existe una cuenta con ese correo' };
```

- `data: { full_name, ...buildConsentMetadata() }` guarda el **nombre** y la **constancia del consentimiento** (`accepted_terms_at`, `accepted_terms_version`) en los metadatos del usuario.
- `identities?.length === 0`: con confirmación activa, **registrar un correo que ya existe no da error** (Supabase **no revela** qué correos tienen cuenta); devuelve un usuario sin identidades. La app lo detecta y avisa.
- `resetPasswordForEmail` «siempre funciona» aunque el correo no exista, **por la misma razón** (no revelar cuentas).

11.4 Confirmar correo y recuperar contraseña: por qué existe `web-auth/` (D-014)

Primer diseño (falló): el correo traía `{{ .ConfirmationURL }}` con redirección a la app (`neurotrackem://…` o `exp://IP:8081`). Fallaba por tres motivos: (1) **PKCE** ata el enlace al **dispositivo que pidió el correo**; (2) **Expo Go no registra** el esquema `neurotrackem://`; (3) el visor de Gmail **no abre esquemas propios** (el usuario veía una ventana negra).

Solución: las plantillas (`supabase/email-templates/`) enlazan a una **página estática** en **Vercel** (`web-auth/`): `{{ .SiteURL }}/confirmar|restablecer?token_hash={{ .TokenHash }}&type=…`. La página:

- llama a **`verifyOtp` al tocar el botón** (no al abrir la página, para que los **revisores automáticos de correo** no gasten el enlace),
- cambia la contraseña con `updateUser` y cierra **todas** las sesiones con `signOut({ scope: 'global' })`,
- **no guarda sesión** (`persistSession: false`),
- recibe la URL y la clave de Supabase por **variables de entorno de Vercel** (`build.js` genera `config.js`, ignorado por Git),

- va **siempre en modo claro** (decisión del usuario; el modo oscuro automático la dejaba negra).

Funciona **desde cualquier dispositivo**. Costo: hay un componente extra que mantener. Plan B descartado por ahora: código de 6 dígitos con `verifyOtp` dentro de la app.

> Nota de código: `auth/api.ts` todavía pasa `emailRedirectTo: Linking.createURL(...)` y existen `src/app/correo/*` y `deep-link.ts` como **respaldo del flujo anterior**; las plantillas ya no los usan. Si te lo preguntan: «quedaron como respaldo, no se usan en el flujo vigente».

11.5 Consentimiento y datos legales (D-015)

- Son **datos de salud, sensibles** según la **Ley Estatutaria 1581 de 2012** (Colombia).
- Al registrarse la persona marca una **casilla** y se guarda `accepted_terms_at` y `accepted_terms_version` (`LEGAL_VERSION` en `features/legal/documents.ts`). **Si se cambia el texto legal hay que subir `LEGAL_VERSION`.** Los metadatos los envía el cliente: son «constancia razonable, no prueba infalible» (alternativa más sólida: tabla de consentimientos con trigger).
- Documentos: términos y privacidad (`src/app/legal/[doc].tsx`), con contacto, terceros (Supabase, Gmail, Vercel), transferencia internacional, retención, derechos con plazos (10/15 días hábiles), mayoría de edad y sección «Médico tratante». **No es asesoría legal.**
- **Pendiente conocido:** el responsable del tratamiento no tiene domicilio ni teléfono (Decreto 1377 de 2013, art. 13).

11.6 Sesión y cierre

- **Arranque:** `bootstrapSession` → `getActiveSession()` → `supabase.auth.getSession()` (lee la sesión cifrada, la refresca si hace falta).
- **Cerrar sesión** (`cuenta.tsx`): `dispatch(logout())` → `supabase.auth.signOut()`; el listener `onAuthStateChange` de `_layout.tsx` detecta `SIGNED_OUT` y despacha `sessionEnded()` + `resetAll()`; la pantalla hace `router.replace('/')`.
- **Borrar cuenta:** `removeAccount` → RPC `delete_my_account` → `signOut({ scope: 'local' })` (**local**, porque el usuario ya no existe y un cierre global fallaría).

---

12. Lógica de dominio pura y pruebas (Jest)

12.1 Por qué «lógica pura» (la idea)

Una **función pura** devuelve siempre lo mismo para los mismos argumentos y **no tiene efectos** (ni red, ni UI, ni reloj oculto). El proyecto saca las reglas a `logic.ts` / `calendar.ts` / `summary.ts` **sin imports de React ni de Supabase**. Beneficios: se **prueban en milisegundos** con Jest, se reutilizan entre pantallas y no dependen del emulador. De ahí salen las **124 pruebas**.

12.2 Las suites (12) y qué prueban

Suite	Pruebas	Qué cubre
`features/doctor/__tests__/logic-test.ts`	27	Vigencia/caducidad de acceso, comparación de períodos con diferencias neutras, constancia de registro, notas
`features/audio/__tests__/` (3 archivos)	18 + 11 + 5 = **34**	Guiones de voz, qué se narra en cada momento, elección de voz en español, partición de textos largos
`features/progress/__tests__/logic-test.ts`	14	Rangos de período, buckets, agregación, insights, titular

Suite	Pruebas	Qué cubre
`features/activities/__tests__/logic-test.ts`	**22**	Generación de rondas de Memoria/Atención/Movimiento/**Lógica**, verificación de respuestas, `nextLevel`
`features/legal/__tests__/legal-test.ts`	7	Términos y privacidad (versión, secciones)
`features/symptoms/__tests__/` (2)	6 + 3 = **9**	Grid del calendario; resumen de síntomas
`features/auth/__tests__/` (2)	3 + 5 = **8**	Validación de correo; análisis de enlaces
`features/about/__tests__/content-test.ts`	3	Contenido «no diagnostica»
Total	**124**	12 suites

Se ejecutan con `npm test` (`jest` con preset `jest-expo`, configurado en `package.json`).

12.3 Código de dominio que debes saber explicar

a) Subida de nivel por precisión — `features/activities/logic.ts`

```
/** RF-003: la precisión decide el nivel, no el tiempo de respuesta. */
export function nextLevel(currentLevel: number, maxLevel: number, roundsCorrect: number): number {
  if (roundsCorrect >= ROUNDS_TO_LEVEL_UP && currentLevel < maxLevel) return currentLevel + 1;
  return currentLevel;
}
```

`ROUNDS_TO_LEVEL_UP = 2` (de 3 rondas). Observa que **no recibe tiempo**: la velocidad **no puede** influir. Esto implementa la promesa §V-B: «evitar que la velocidad sea el único criterio».

b) Generadores de rondas

- **Memoria:** `length = Math.min(level, 5) + 2` → nivel 1 = 3 figuras, nivel 5 = 7. Se eligen con `pickRandom(ALL_SHAPES)`. `checkMemoriaAnswer` exige **misma longitud y mismo orden**.
- **Atención:** `optionCount = Math.min(level, 5) + 4` → 5 a 9 opciones; una es el «intruso» (`isOdd`).
- **Movimiento:** `stepCount = Math.min(level, 4) + 2` → 3 a 6 pasos, tomados **mezclados** del banco `MOVIMIENTO_STEP_POOL` (barajado de Fisher-Yates).
- **Lógica (series numéricas):** `generateLogicaRound(level, rng = Math.random)` recibe el generador de números **como parámetro** (inyección de dependencia) para poder **probarla de forma determinista**. Niveles: 1 = suma de 1/2/5/10, 2 = suma de 2 a 9, 3 = resta, 4 = ×2 o ×3, 5 = dos pasos alternados (p. ej. +2, +5). `buildLogicaOptions` arma **4 opciones distintas y no negativas**: la respuesta y 3 distractores cercanos.

c) Calendario — `features/symptoms/calendar.ts`

- `buildMonthGrid(year, month, logs)`: agrupa los registros por **día** y calcula la **severidad más alta del día** con `SEVERITY_RANK = { leve: 1, moderado: 2, fuerte: 3 }` y un `reduce`.
- `leadingBlankCount`: `(firstWeekday + 6) % 7` → columnas vacías antes del día 1 con la **semana empezando en lunes** (`getDay()` da 0 = domingo).
- `month` es **0-indexado** (como `Date`).

d) Progreso — `features/progress/logic.ts`

- `getPeriodRange(period, reference)` devuelve `[start, end)` para `semana` (lunes a domingo), `mes` (1.º al último) y `tresMeses` (mes de referencia y los 2 anteriores).
- `getPreviousPeriodRange` mueve la referencia un período atrás → es la base de **comparar actual vs. anterior** (RF-005).
- `aggregateBuckets` cuenta, por grupo, **días distintos con síntomas** (con un `Set` de fechas) y **sesiones de actividad**.
- `MIN_DISTINCT_DAYS_FOR_INSIGHTS = 10`: con menos días registrados la app muestra una tarjeta de **«datos insuficientes»** en vez de conclusiones engañosas.
- `fetchProgressData` pide a la base el rango **desde el inicio del período anterior hasta el fin del actual**, con `Promise.all` (dos consultas en paralelo: síntomas y sesiones).

e) Médico — `features/doctor/logic.ts`

- `compareSymptoms`: por tipo de síntoma, **días distintos** en el período actual vs. el anterior; produce textos **neutros** («3 días más que el mes anterior», «Igual que…»). **Nunca** «mejor/peor» ni alertas (RT-004).
- `isLinkExpired` / `isLinkActive`: `aceptado` y con `expiresAt` pasado = vencido. **Esto refleja en la app lo que ya impone la base de datos** (la base es la que realmente corta el acceso).
- `expiresAtFromDuration`: 3/6/12 meses o `null` (sin fecha).

12.4 Cómo se escribe una prueba (por si piden agregar una)

```
describe('generateMemoriaRound', () => {
  test('la secuencia crece con el nivel (nivel 1 → 3, nivel 5 → 7)', () => {
    expect(generateMemoriaRound(1).sequence).toHaveLength(3);
    expect(generateMemoriaRound(5).sequence).toHaveLength(7);
  });
});
```

`describe` agrupa; `test` es un caso; `expect(...).toHaveLength/toBe/toEqual` verifica. **Ejecutar solo una suite:** `npx jest src/features/activities`.

12.5 Qué NO se probó (sé honesto)

- No hay pruebas de **UI** ni de **integración con Supabase** (se verificó manualmente en emulador/celular y con capturas).
- Sin prueba unitaria del `condition` de `fetchAreaProgress` ni del trigger `patient_name`.
- Criterio de D-004: se prueba **lógica pura**; los módulos que solo tienen `api.ts` (llamadas a Supabase) no tienen lógica que valga testear sin *mocks*.

---

13. Los módulos de la app, uno por uno

> Para cada módulo: **qué hace · archivos · flujo · decisiones · qué pregunta esperar.**

13.1 Selección de perfil y acceso (Samuel)

- **Archivos:** `src/app/index.tsx`, `src/components/profile-option.tsx`, `src/app/(auth)/acceder.tsx`, `recuperar.tsx`, `src/components/text-field.tsx`, `src/features/auth/*`, `src/store/slices/auth-slice.ts` y `profile-slice.ts`.
- **Flujo:** elegir perfil (`dispatch(profileSelected(...))`) → si no hay sesión, `acceder` (pestañas *Iniciar sesión* / *Crear cuenta*) → `login`/`register` → al estar `authenticated`, `Redirect` al inicio del perfil.

- `ProfileOption` es un `Pressable` con `accessibilityRole="radio"`, `accessibilityState={{ checked: selected }}`, `accessibilityLabel={title}` y `accessibilityHint={description}`, dentro de un contenedor con `accessibilityRole="radiogroup"`. El estado seleccionado se indica con **borde + ícono de check + color** (no solo color).
- `TextField`**:** etiqueta, ícono, **borde doble-codificado en error** (borde + ícono + texto, RNF-006), botón mostrar/ocultar contraseña, y un banner de error con `accessibilityRole="alert"` y `accessibilityLiveRegion="assertive"`. Acepta `inputRef` para **mover el foco** al primer campo inválido.
- **Bug real:** el error de `register` solo se mostraba en modo «login»; en «Crear cuenta» parecía que el botón no hacía nada. Se corrigió para mostrarlo en ambos.

13.2 Síntomas y calendario (Samuel) — RF-001, RF-002

- **Archivos:** `src/app/registrar-sintoma/{_layout,paso-1,paso-2,paso-3}.tsx`, `src/app/(paciente)/calendario.tsx`, `src/features/symptoms/*`, `src/store/slices/symptoms-slice.ts`, migración `20261108…symptom_logs.sql`.
- **Tipos de síntoma:** fatiga, visión borrosa, dolor, debilidad/entumecimiento, equilibrio, otro. **Severidad:** leve / moderado / fuerte (con **barras de 1, 2 y 3** + la palabra).
- **Decisiones:** asistente de **3 pasos** (un paso por pantalla = baja carga cognitiva, RNF-008); nota **opcional** de máx. 300 caracteres; botón **Dictar con voz** (REC-001); `Redirect` si el borrador está incompleto; **caché por mes** en el slice.
- **Calendario:** grid del mes con **indicador de severidad por día** (barras, no solo color), detalle del día, botón flotante «Registrar», navegación entre meses con botones con `accessibilityLabel` («Mes anterior», «Mes siguiente»); cada día es un `Pressable` con `accessibilityState={{ selected }}`.
- **Pregunta probable:** *«¿Por qué mandan `occurred_at` desde el cliente?»* → para que el registro caiga en el «hoy» que ve el paciente; si fuera el reloj del servidor, un desfase horario lo movería de día.

13.3 Actividades (Mariana) — RF-003, RF-004

- **Archivos:** `src/app/(paciente)/actividades.tsx`, `src/app/actividad/[area].tsx` (modal con las 4 áreas), `src/features/activities/{logic,constants,types,api}.ts`, componentes `area-tile`, `shape-button`, `feedback-badge`, `session-header`, `src/store/slices/activities-slice.ts`, migraciones `activity_progress`, `activity_sessions`, `activity_area_logica`.
- **Pantalla Actividades:** tarjeta «**Tu sesión de hoy**» (sugiere el área con **menor nivel**) y una grilla de 4 áreas con su nivel («Nivel 3 de 5») en barras segmentadas.
- **Las 4 áreas:**
  - **Memoria:** se **memoriza** una secuencia de figuras **sin cronómetro** («Listo, continuar» a su ritmo) y se repite tocando 4 botones grandes. Tras cada toque hay **retroalimentación inmediata** (borde + ícono ✓/✕ + banner «¡Bien!») y **háptica**. **Falla rápido** (D-009). Botón **«Ver la secuencia otra vez»** sin penalización.
  - **Atención:** encontrar la **figura distinta** entre 5 y 9 opciones; sin tiempo.
  - **Lógica:** serie numérica; elegir entre 4 opciones; al responder se muestra **la regla** (D-018).
  - **Movimiento:** 3–6 pasos guiados («Levanta ambos brazos…»), cada paso se marca **a su ritmo**; es **«apto con temblor»**; sube de nivel por **participación** (completar la sesión), no por precisión.
- **Flujo técnico:** `useEffect` arranca `sessionStarted(area)`; al llegar a `completada` otro `useEffect` despacha `recordActivitySession({ area })` (historial) y, al cerrar, `persistAreaLevel({ area, level:`

`session.newLevel })` (guarda el nivel con **`upsert`** en `activity_progress`, `onConflict: 'patient_id,area'`).

- **Háptica:** `Haptics.notificationAsync(Success|Error)` al tocar opciones; `Haptics.selectionAsync()` al marcar un paso; éxito al terminar. **Nunca es el único canal** (RNF-006/009).
- **Reducir animaciones** (`useReduceMotion`): apaga el autoborrado a los 900 ms del resaltado en Memoria.
- **Bug real y solución de lint:** el resaltado transitorio que se borra solo **no puede** hacerse con `setState` dentro de un `useEffect` (la regla de ESLint `react-hooks/set-state-in-effect` lo rechaza); se mueve el `setState` + `setTimeout` al **manejador del evento** y se limpia el temporizador en un efecto de *cleanup*.
- **Por qué «Lógica» estuvo bloqueada y luego se hizo:** D-009 la dejó bloqueada por no tener diseño; D-018 la implementó porque la propuesta (§V-B) promete «memoria, atención y lógica» y una tarjeta «Próximamente» prometía algo que nunca ocurría.

13.4 Progreso (Mariana) — RF-005

- **Archivos:** `src/app/(paciente)/progreso.tsx`, `src/features/progress/{logic,constants,types}.ts`, `src/store/slices/progress-slice.ts`.
- **Qué muestra:** selector **Semana / Mes / 3 meses**, navegación anterior/siguiente, **gráfico de barras** (días con síntomas y actividades hechas) **o tabla** («Ver como tabla»), hasta **3 «insights»** (con ícono + texto, nunca solo color) y la tarjeta de **datos insuficientes**; botón «Preparar resumen para mi consulta» usa `Share` nativo.
- **Accesibilidad del gráfico (bug real con TalkBack):** las barras eran decorativas y TalkBack **no podía seleccionarlas**. Ahora cada barra es un `Pressable` con `accessibilityRole="button"`, etiqueta («Días con síntomas, Semana 2: 3»), `accessibilityState.selected` y una línea de detalle con `accessibilityLiveRegion`. **La tabla es la alternativa** accesible (las barras de la vista semanal miden ~24 dp, no 48: cumple el mínimo de WCAG 2.2 AA de 24 px).
- **Rendimiento:** muchos `useMemo` encadenados (`reference → currentRange → boundaries → buckets → insights`); un `useEffect` vuelve a pedir datos cuando cambia `period` o `referenceISO`.
- **Aviso fijo:** «Esto es un resumen de registros, no un diagnóstico».

13.5 Cuidador (Amelyn) — RF-006, RF-008, RF-012

- **Archivos:** `src/app/(cuidador)/{inicio,acompanar,observaciones,recordatorios,apoyo}.tsx`, `src/app/vincularme.tsx`, `src/app/nueva-observacion.tsx`, `src/app/permisos.tsx` (lado paciente), `src/features/caregiver/*`, `src/features/reminders/*`, `src/store/slices/caregiver-slice.ts`, `reminders-slice.ts`, migraciones `caregiver_links`, `caregiver_observations_and_access`, `reminders`, `harden_links_and_account`, `caregiver_patient_name`.
- **Vincular:** el **paciente** abre «Cuidadores» (`permisos.tsx`), ve su **código** y lo comparte (botón **Compartir** con `Share`); el **cuidador** lo escribe en `vincularme.tsx` (usa `TextField`, 6 caracteres) → queda «Solicitud enviada»; el paciente **acepta** con 3 `Switch` de permiso o rechaza; puede «**Quitar acceso**» (con confirmación).
- **Inicio del cuidador:** 3 estados — sin vincular (CTA «Vincularme»), solicitud pendiente, vinculado («Acompañas a », resumen de la semana).
- **Acompañar:** resumen semanal por área, nivel por área, **«Síntomas esta semana»** (vía `shared_symptom_logs`) y un botón de **mensaje de ánimo** con 3 opciones.
- **Observaciones:** bitácora **privada** agrupada por día (categorías: ánimo, movilidad, fatiga, medicación, otro); el paciente **nunca** la ve (política RLS sin `select` para el paciente; D-010).
- **Recordatorios:** el cuidador **crea** (categoría + hora entre 4 combinaciones fijas: hoy mañana/tarde/noche, mañana por la mañana — se evitó añadir un selector de fecha nativo y una

dependencia nueva); el paciente los ve en **«Hoy»** de su inicio y los marca **«Hecho»** con **ícono de check + vibración** (alternativa visual+háptica a un sonido; RF-008).
- **Decisión D-010:** código corto (no QR ni enlace profundo: sin cámara ni *deep linking*); `caregiver_email/name` **copiados en la fila** (no join a `auth.users`, imposible por RLS); sin toggle «Mis notas» (no existe esa funcionalidad).
- **Consecuencia aceptada de D-010:** el nombre/correo del cuidador que ve el paciente queda «congelado» al momento de la solicitud.

13.6 Apoyo (Amelyn) — RF-009, RF-010

- **Archivos:** `src/app/(paciente)/apoyo.tsx`, `src/app/(cuidador)/apoyo.tsx`, `src/app/apoyo/[id].tsx`, `src/features/support/{constants,types,crisis-section,guide-list}.tsx`.
- **Contenido: Línea 106 (Bogotá, 24 h, gratuita)** y **Emergencias 123** con botón «Llamar» (`Linking.openURL('tel:…')`); **4 guías por perfil** con filtro Todo/Leer/Escuchar/Ver y pantalla de detalle.
- **Decisión de alcance (RF-010): todo es estático** (bundleado, sin red, sin audio/video grabado). Las guías «Escuchar/Ver» se entregan **desde el inicio como transcripción completa en texto**, siempre visible: cumple «subtítulos/transcripción» sin depender de media real.

13.7 Perfil del médico (extra) — REC-003, D-016

- **Archivos:** `src/app/(medico)/{pacientes,vincular-paciente,paciente/[id]}.tsx`, `src/app/mi-medico.tsx` (lado paciente), `src/features/doctor/*`, `src/store/slices/doctor-slice.ts`, migración `20261115…doctor_links.sql`.
- **Qué ve el médico:** constancia de registro («N de M días»), **comparación** de síntomas del período contra el anterior con diferencias **neutras**, intensidad (barras + palabra), actividades y **notas solo si el paciente las autorizó**.
- **Principios (D-016):** solo lectura (sin chat, sin escribir nada); **tendencias, nunca alertas**; aviso fijo «no es una evaluación clínica»; **caducidad**; «último acceso» visible para el paciente; registro profesional **autodeclarado y marcado «no verificado»** (RETHUS fuera de alcance).
- **Accesibilidad:** cada permiso de «Mi médico» es un `Pressable` con rol `switch` y el `Switch` visual se oculta al lector (`importantForAccessibility="no-hide-descendants"`) para **no anunciarlo dos veces**.
- **Diseño:** se combinó el diseño propio con lo objetivo de **Stitch** y se **descartó** lo que inventaba datos que la app no recoge (edad, diagnóstico, «nivel de energía», expediente), el tono clínico y «cifrado de extremo a extremo» (es TLS).

13.8 Voz: dictado y lectura en voz alta — REC-001, REC-002, D-013

- **Archivos:** `src/features/audio/{dictation-button,speak-button,speech,voice,scripts,screen-scripts,use-activity-narration}`.
- **Dictado (REC-001):** el botón **no reconoce voz**; **enfoca el campo** y explica «toca el micrófono del teclado y habla». Razón: Expo SDK 58 **no trae reconocimiento de voz** y las librerías externas exigen build nativa (Expo Go no las admite). Bonus de privacidad: el audio lo procesa **el teclado del teléfono**, no la app (RNF-012).
- **Lectura en voz alta (REC-002):** `expo-speech` con la **voz del sistema** (funciona **sin conexión**, sin enviar nada). `chooseSpanishVoice` prefiere **es-CO**, luego es-419, es-US, es-MX y cualquier español; velocidad **0,9** (baja la fatiga cognitiva); `splitForSpeech` parte textos largos **por frases** (el motor rechaza textos > ~4000 caracteres; margen de 3500).

- `speak()` **descarta lecturas obsoletas** con un contador (`latestRequest`): dos toques seguidos no se pisan.
- **El interruptor «Lectura en voz alta» manda sobre todo:** `SpeakButton` **no se pinta** si está apagado (`if (!enabled) return null`). Se cambió el primer diseño (botón activo con interruptor apagado) porque **era confuso**; por defecto **apagado** (quien usa TalkBack no la necesita y se pisarían).
- **Con TalkBack activo, Android da prioridad a TalkBack** y corta la voz de la app: es el comportamiento correcto y **no se intentó «arreglar»**.
- Los guiones (`scripts.ts`) y `narrationFor` son **funciones puras** (34 pruebas).

13.9 Pantalla de Accesibilidad — D-011

`src/app/accesibilidad.tsx`, con **6 controles reales** (todos con efecto verificable): **Apariencia** (claro/oscuro/automático), **Brillo de la app** (Del teléfono/Bajo/Muy bajo, `expo-brightness`, solo mientras la app está abierta), **Tamaño de texto** (normal/grande/extra grande), **Alto contraste**, **Reducir animaciones**, **Lectura en voz alta**; más **«Restablecer configuración»** (`settingsReset`). Se persisten con `AsyncStorage` (`features/settings/persistence.ts`) y se hidratan al abrir (`hydrateSettings`). **Se descartó** el mock original con «perfiles por diagnóstico» (Dislexia/TDAH/…) por ser el patrón de *accessibility overlay widget* que la comunidad critica, y los botones sin equivalente nativo (puntero grande, resaltar enlaces…): **un control que no hace nada es peor que no tenerlo**.

13.10 Cuenta, legal y «¿Qué es?»

`src/app/cuenta.tsx` (cerrar sesión, **borrar cuenta** con confirmación `Alert`, accesos a Mi médico/Accesibilidad/Términos/Privacidad), `src/app/legal/[doc].tsx`, `src/app/acerca.tsx` + `features/about/` («¿Qué es Neurotrack EM?»: para quién es, funciones, privacidad, preguntas frecuentes y aviso médico con 123/106).

---

14. Accesibilidad técnica a fondo

> **El enunciado cita textualmente:** `accessibilityRole`, `accessibilityLabel`, **manejo de foco**, `lineHeight`, `Pressable`. Esta sección cubre esas cinco cosas con código real. Es la parte que **todos** deben dominar, aunque su módulo sea otro.

14.1 Marco

- **WCAG 2.2 nivel AA** (Web Content Accessibility Guidelines): pautas internacionales. Aunque nacieron para la web, se aplican a apps móviles. **POUR** = **P**erceptible, **O**perable, **C**omprensible (*Understandable*), **R**obusto.
- **TalkBack** = lector de pantalla de **Android** (VoiceOver es el de iOS; no se verificó en iOS). Un lector de pantalla recorre la interfaz **elemento por elemento** (swipe), **lee** el nombre, el rol y el estado, y se activa con doble toque.
- React Native expone la accesibilidad con **props nativas** que se traducen a las APIs de cada sistema; **no se usó ninguna librería externa** (V-D6: «apoyarse en lo que ya incluye el framework»).
- **Cifras del repositorio** (conteo con `grep` sobre `src/`, 2026-11-11): `accessibilityRole` 211 líneas, `accessibilityLabel` 172, `accessibilityLiveRegion` 35, `hitSlop` 43, `testID` 309 referencias.

14.2 Las props, una por una (qué hace cada una y dónde se ve)

Prop	Qué le dice a TalkBack	Ejemplo real en el repo
`accessible`	«Trata este contenedor como **un solo elemento** enfocable» (agrupa a sus hijos)	`ScreenState` (`<View accessible …>`)
`accessibilityLabel`	El **nombre** que se lee. Si no se pone, se lee el texto interno; para **íconos o botones sin texto es obligatoria**	`accessibilityLabel="Mes anterior"` en el calendario; `Mostrar contraseña`/`Ocultar contraseña` en `TextField`
`accessibilityHint`	Una **pista** de lo que ocurrirá al activar, que se lee **después** de la etiqueta	`ProfileOption`: `accessibilityHint={description}`; el enlace «¿Qué es Neurotrack EM?» → «Abre una pantalla con información sobre la app»
`accessibilityRole`	El **tipo** de elemento (botón, encabezado, casilla, interruptor…) para que TalkBack diga «botón», «encabezado» y active los gestos correctos	`button` (101 usos), `header` (55), `alert` (17), `radiogroup`/`radio` (9/9), `link` (8), `progressbar` (4), `checkbox` (2), `tab`/`tablist`, `switch`
`accessibilityState`	El **estado** actual: `selected`, `checked`, `disabled`, `busy`, `expanded`	`ProfileOption`: `{ checked: selected }`; `ScreenState`: `{ busy: kind === 'cargando' }`; días del calendario: `{ selected: isSelected }`
`accessibilityLiveRegion`	«Cuando este contenido **cambie**, **anúncialo** sin que el usuario lo busque» (`polite` = cuando termine de hablar; `assertive` = interrumpe)	Errores de campo (`assertive`); carga (`polite`); resumen de Progreso al cambiar de período
`importantForAccessibility`	Oculta elementos **decorativos o duplicados** del árbol de accesibilidad (`no-hide-descendants`)	El `Switch` visual de «Mi médico», para que no se anuncie dos veces
`accessibilityIgnoresInvertColors`	(iOS) no invertir colores en imágenes	La ilustración de la selección de perfil
`testID`	**No** es accesibilidad: es un identificador estable para **pruebas automatizadas** (en Android se expone como `resource-id`)	`testID="profile-option-paciente"`; se usó para automatizar el emulador

> **Matiz que luce:** `accessibilityRole` en React Native funciona sobre **Android y iOS**; algunos roles (`radiogroup`, `tablist`) agrupan a sus hijos para que el lector anuncie «opción 2 de 3».

14.3 Agrupar para evitar «focos fragmentados»

Problema: una tarjeta con ícono + título + descripción + chip, si cada parte es enfocable, obliga a **cuatro swipes** para leer una sola opción y los fragmentos pierden contexto.

Solución en el repo: el contenedor táctil es **un solo elemento** con la etiqueta completa. `ProfileOption` (`src/components/profile-option.tsx`):

```
<Pressable
  onPress={onPress}
  accessibilityRole="radio"
```

```
  accessibilityState={{ checked: selected }}
  accessibilityLabel={title}
  accessibilityHint={description}
  hitSlop={8}
  style={({ pressed }) => [styles.card, { backgroundColor: …, borderColor: selected ? colors.primary
: 'transparent', opacity: pressed ? 0.85 : 1 }]}>
```

TalkBack lee: **«Soy paciente, botón de selección, no seleccionado. Registra cómo te sientes y haz tus actividades.»** Todo en **un** foco. Si está seleccionada, además del `checked`, la tarjeta cambia **borde + fondo + ícono de check** (nunca solo color).

El contenedor padre (`index.tsx`, `paso-1.tsx`, `paso-2.tsx`) usa `accessibilityRole="radiogroup"` con `accessibilityLabel="Selección de perfil"`/«Tipo de síntoma», así el lector anuncia **cuántas opciones hay**.

14.4 `Pressable` (por qué y cómo)

- **`Pressable`** es el componente táctil **moderno** de React Native: expone el estado `pressed` (función de `style`), `hitSlop`, `android_ripple`, y **todas** las props de accesibilidad. **No se usó** `TouchableOpacity` (verificado: 0 usos): es el componente antiguo, con menos flexibilidad.
- Patrón del repo: `style={({ pressed }) => [base, { opacity: pressed ? 0.85 : 1 }]}` → retroalimentación visual **inmediata** al tocar.
- **`hitSlop`** = aumenta el área **táctil** más allá de los límites visibles **sin cambiar el diseño**. Ejemplo: íconos de 32 dp con `hitSlop={8}` → zona de toque de 48 dp.

14.5 Manejo de foco (¡lo cita el enunciado!)

Qué es: controlar **a qué elemento va el foco** (del teclado o del lector) en momentos clave, para que una persona con TalkBack **no tenga que buscar** el error.

Dónde está en el código: `src/app/(auth)/acceder.tsx`, función `validar()`:

```
const primero: [string, RefObject<TextFieldInput | null>][] = [
  ['nombre', nombreRef], ['correo', correoRef], ['password', passwordRef], ['confirmPassword',
confirmRef],
];
const conError = primero.find(([campo]) => errores[campo]);
conError?.[1].current?.focus();
```

- Se mantienen **referencias** (`useRef`) a cada `TextInput` (a través del prop `inputRef` de `TextField`).
- Al enviar con errores, **se enfoca el primer campo inválido**: TalkBack **salta ahí** y lee su etiqueta **y el aviso de error** (`accessibilityHint={hasError ? errorText : undefined}` + banner con `accessibilityRole="alert"`).
- Mismo patrón en `reenviarConfirmacion()` (enfoca el correo si falta) y en el **dictado**: `DictationButton` llama a `noteInputRef.current?.focus()` para abrir el teclado con el micrófono.
- **Orden lógico de lectura:** como el orden de los elementos en el JSX coincide con el visual (arriba→abajo), TalkBack los recorre en un **orden lógico** sin configurar nada extra; solo se evitan posicionamientos absolutos engañosos.
- **Modales y rutas:** el asistente es un modal que se cierra con `router.dismissTo`/`router.dismissAll`.

14.5 bis Anunciar cambios (regiones vivas)

Sin `accessibilityLiveRegion`, un lector **no se entera** de que algo cambió (p. ej. «Cargando…» → datos, o un error que aparece). Usos: `ScreenState` (carga `polite`, error `assertive`), banner de error de `TextField` y de `paso-3` (`assertive`), resumen de Progreso al cambiar de Semana/Mes (`polite`), detalle de la barra seleccionada del gráfico.

14.6 Texto escalable y `lineHeight` (¡lo cita el enunciado!)

El problema técnico (explícalo con calma):

- El usuario puede aumentar el **tamaño de fuente del sistema** (Ajustes de Android). React Native **escala `fontSize`** automáticamente según esa preferencia (`<Text>` tiene `allowFontScaling` activo por defecto).
- **Pero NO escala `lineHeight`** si lo defines como número fijo. Resultado: con fuente grande, la **altura de línea queda igual y el texto se recorta o se superpone** (*clipping*). Lo encontró la auditoría de accesibilidad (RNF-002).

La solución (`src/components/themed-text.tsx`):

```
const { lineHeight, fontSize, ...typeScale } = TypeScale[variant];
const scaledFontSize = fontSize ? fontSize * appTextScale : undefined;
const scaledLineHeight = lineHeight ? lineHeight * appTextScale * PixelRatio.getFontScale() :
undefined;
```

- `appTextScale` = multiplicador de **la propia app** (Accesibilidad → «Tamaño de texto»: normal 1, grande 1,15, extra grande 1,3).
- `PixelRatio.getFontScale()` = factor de **fuente del sistema** (1,0 / 1,15 / 1,3 …).
- **`fontSize`** se multiplica solo por `appTextScale` porque **el sistema ya aplica su propio factor** automáticamente; **`lineHeight`** necesita **ambos** porque React Native no le aplica el del sistema.
- **Resultado:** los dos controles se **combinan** (app × sistema) en vez de pisarse, y la altura de línea acompaña al tamaño real.
- **Por qué un componente `ThemedText`: todos** los textos de la app lo usan, así que arreglar el `lineHeight` **en un solo lugar** lo arregla **en toda la app**. Define variantes (`bodySm … headlineLg`) tomadas de `TypeScale`.
- En diseño de layout se usan `minHeight` y `flexShrink` en lugar de alturas fijas, para que el texto crezca sin romper.
- **Matiz:** `maxFontSizeMultiplier={1.2}` se usa en un único sitio (`doctor/components.tsx`, avatar de iniciales) para que el texto **dentro de un círculo fijo** no desborde; es una excepción deliberada.

Tipografía: `src/theme/typography.ts` define **Atkinson Hyperlegible Next** (regular 400 y bold 700; cargadas con `useFonts` en `_layout.tsx`). Escala: bodySm 15/22, bodyMd 17/26, bodyLg 19/28, labelSm 13/18, labelMd 15/20, labelLg 17/24, headlineSm 20/28, headlineMd 24/32, headlineLg 28/36 (tamaño/altura de línea).

14.7 Contraste (WCAG 1.4.3 y 1.4.11)

La fórmula (la implementa `scripts/validar-paleta.mjs`):

1. Se convierte cada canal RGB (0–255 → 0–1) a **lineal** (`lin`): `c ≤ 0,04045 ? c/12,92 : ((c+0,055)/1,055)^2,4`.
2. **Luminancia relativa** `L = 0,2126·R + 0,7152·G + 0,0722·B` (el ojo es más sensible al verde).

3. **Razón de contraste** `(L_claro + 0,05) / (L_oscuro + 0,05)`. Va de **1:1** (iguales) a **21:1** (blanco/negro).

Umbrales AA: 4,5:1 para texto normal; **3:1** para texto grande (≥ 24 px, o 18,66 px en negrita) y para **componentes de interfaz y gráficos** (íconos, bordes, 1.4.11).

Resultados del script: 18 pares texto/fondo e ícono/borde, en claro y oscuro, **todos cumplen**. Ejemplos: texto principal 16,3:1 (claro) y 14,2:1 (oscuro); texto sobre el botón principal 11,1:1 y 6,7:1.

Cómo está organizado el color (`src/theme/colors.ts`): roles de Material Design 3 (`primary`/`onPrimary`, `surface`/`onSurface`, `secondaryContainer`/`onSecondaryContainer`…). **Regla M3:** un rol `onX` **solo** se pinta sobre su par `X`; por eso los pares están validados. Hay dos esquemas (`LightColors`, `DarkColors`) y un tercero derivado (`applyHighContrast`).

Dos defectos reales de contraste (¡buena historia para el profesor!):

1. Se usaban roles `onSecondaryFixed` **sobre** `secondaryContainer`: en oscuro el contraste caía a **1,00–1,82:1** (invisible). Los roles `Fixed` están pensados para pintarse sobre su propio contenedor `Fixed`. **Arreglo:** usar `onSecondaryContainer` (que sí cambia por esquema).
2. El texto de los **botones principales** usaba `onPrimary` **sobre** `primaryContainer` en oscuro (`#00363f` sobre `#223a40`, **≈ 1,1:1**), en unos 25 archivos. **Arreglo (D-017):** usar `colors.primary` como relleno (6,7:1 oscuro, 11,1:1 claro) **sin tocar la paleta** (D-012), con el script como verificación. Quedó visto en emulador (`artifacts/t11/01,02,05,07`).

Alto contraste (`applyHighContrast`): sobrescribe pares a negro/blanco **puros** (excepción deliberada a RNF-007, porque es un modo que el usuario **elige a propósito**).

14.8 No depender solo del color (§V-D3, 1.4.1) y daltonismo

Cómo se cumple: todo estado tiene **color + ícono/forma + texto**.

- **Severidad:** barras de **1, 2 o 3** (llenas o con contorno) + palabra «Leve/Moderado/Fuerte» (`severity-bars`, calendario).
- **Actividades:** insignia **✓/✕ + texto** («¡Bien!», «Vamos, tú puedes…»).
- **Permisos:** «Sí/No»; **comparaciones:** flechas y texto («3 días más…»); **validación de campos:** borde + ícono + texto.
- **Evitar rojo-verde** para «mejor/peor» (la propuesta lo prohibía).

Validación contra daltonismo (`validar-paleta.mjs`): simula **protanopia, deuteranopia y tritanopia** multiplicando cada color (en RGB lineal) por las **matrices de Machado et al. (2009)** y mide la diferencia de color **ΔE (CIE76)** en espacio **Lab** entre los colores de estado. Resultado: acierto/error **ΔE ≥ 23** en las tres simulaciones; severidad frente al fondo **ΔE ≥ 70**. **Única pareja bajo ΔE 15:** primario/secundario en oscuro (11 y 14); **no es señal única** (el estado «seleccionado» se anuncia con `accessibilityState` y cambia el relleno, 8,8:1).

14.9 Ergonomía táctil (§V-C)

- `MinTouchSize = 48` (dp) en `src/theme/spacing.ts` (WCAG 2.2 pide mínimo 24 px —criterio 2.5.8—; las guías de Material piden **48 dp** y las de Apple **44 pt**; el proyecto usa el mayor).
- Se aplica con `minHeight: MinTouchSize` / `minWidth` en botones; los principales miden **56 dp**.
- `hitSlop` compensa íconos pequeños. La auditoría encontró **2 excepciones** (botón «Cuidadores» a 40 dp; `48` literal en el botón de llamada de crisis) y se corrigieron (`hitSlop={8}` y el token).

- **Sin gestos complejos:** solo **toques simples**; sin *swipe*, *drag*, pellizco ni mantener pulsado. **Sin límite de tiempo** en actividades.
- **Barras del gráfico de Progreso:** ~24 dp de ancho en la vista semanal (cumple el mínimo de WCAG 2.2 de 24 px, no los 48 dp del resto); **la tabla es la alternativa**.

14.10 Modo oscuro, bajo brillo y alto contraste (§V-D4)

- `src/theme/index.ts`: `useAppColorScheme()` decide el esquema: si la preferencia guardada es `light`/`dark`, **manda**; si es `auto`, usa el del sistema (`useColorScheme` de React Native). **Es independiente del sistema** (lo pedía la propuesta).
- `useThemeColors()` devuelve el diccionario de colores del esquema activo **y** aplica `applyHighContrast` si está activado.
- `DarkColors`: paleta real de Stitch (D-012), **sin blanco ni negro puros**, con **escala de elevación**: las superficies se **aclaran** a mayor elevación; se añadió un borde de 1 px a las tarjetas para que no desaparezcan.
- **Excepción deliberada a M3:** `tertiaryFixed` y compañeras **sí cambian** en oscuro (si no, las tarjetas grandes se ven como un recorte claro sobre fondo oscuro).
- **Bug real:** la barra de pestañas nativa no heredaba el tema (se veía en gris «Material You»); se pasaron los colores por props.

14.11 Alternativas al audio y a la vista (§V-A y §V-D5)

Necesidad	Solución
Alerta que no dependa del sonido	Ícono de check + **vibración** (`expo-haptics`) al completar un recordatorio
Contenido audiovisual accesible	Guías «Escuchar/Ver» **como transcripción completa en texto**
Persona con baja visión o ceguera	TalkBack; **Lectura en voz alta** (`expo-speech`) opcional
Escribir cuesta (temblor, fatiga)	**Dictar con voz** (micrófono del teclado)
Errores	Siempre **texto + ícono** (y `accessibilityRole="alert"`)

14.12 Cómo se verificó en dispositivo

- **TalkBack en celular Android real** (2026-11-11, modo oscuro): **14 capturas** con el foco verde del lector en `artifacts/talkback/` (nota en su `README.md`): selección de perfil, acceso, inicio, asistente de síntomas con dictado, calendario, actividades, accesibilidad y progreso (perfil paciente).
- **Auditoría de código** con el agente `accessibility-reviewer` (2026-11-09): grep sistemático de props y cálculo de contraste sobre ~29 pares → **2 defectos reales** (`lineHeight`, contraste en oscuro) y **2 áreas táctiles** bajo el mínimo, todo corregido; confirmó RNF-004, 006, 008, 010, 011.
- **Hallazgo con TalkBack real:** las barras del gráfico de Progreso no eran seleccionables (ya arreglado).
- **Limitación:** no hay capturas con **fuente del sistema ampliada**, ni TalkBack en cuidador/médico, ni iOS.

14.13 La matriz trazable (Tabla 4 del documento final) resumida

#	Mejora prometida	Dónde está
1	Alternativa visual + háptica a alertas	`(paciente)/inicio.tsx`, `actividad/[area].tsx`
2	Transcripción/subtítulos	`features/support/constants.ts`, `apoyo/[id].tsx`
3	`accessibilityLiveRegion`	`screen-state.tsx`, `text-field.tsx`, `progreso.tsx`

#	Mejora prometida	Dónde está
4	Instrucciones cortas, pasos simples	`registrar-sintoma/paso-1..3.tsx`, `InstructionCard`
5	Dificultad progresiva sin velocidad	`activities/logic.ts`
6	Áreas táctiles amplias	`MinTouchSize`, `hitSlop`
7	Texto escalable sin recortes	`themed-text.tsx`
8	Un elemento principal por pantalla	patrón título + 1 acción + `ScreenState`
9	Color + ícono + texto	`calendario.tsx`, `severity-bars.tsx`, `ShapeButton`
10	Paleta validada en 3 tipos de daltonismo	`src/theme/colors.ts` + `scripts/validar-paleta.mjs`
11	Bajo brillo, sin blanco puro	`DarkColors`, `accesibilidad.tsx`
12	Apariencia independiente del sistema	`features/settings/persistence.ts`
13	Lector de pantalla, etiquetas, orden lógico	`accessibility*` en `src/app/**`, `src/components/**`
14	Dictado para síntomas	`features/audio/dictation-button.tsx`
15	Versión auditiva	`expo-speech`, «Lectura en voz alta»

14.14 Preguntas típicas de accesibilidad

- *«¿Qué hace `accessibilityRole`?»* Le dice a TalkBack **qué tipo de elemento es** («botón», «encabezado»…) y habilita su comportamiento. Ej.: `accessibilityRole="header"` en los títulos permite navegar por encabezados.
- *«¿Diferencia entre `accessibilityLabel` y `accessibilityHint`?»* Label = **qué es** («Mes anterior»); hint = **qué pasa al usarlo** («Abre una pantalla con información»).
- *«¿Qué pasa si un botón es solo un ícono?»* Sin `accessibilityLabel` TalkBack no tiene qué leer: es obligatoria.
- *«¿Por qué escalaron `lineHeight` a mano?»* Porque RN escala `fontSize` pero no un `lineHeight` numérico; sin escalarlo el texto se recorta.
- *«¿Cómo manejan el foco?»* Refs + `.focus()` al primer campo inválido en `acceder.tsx`; `accessibilityLiveRegion` para anuncios; `Redirect` para flujos incompletos.
- *«¿Qué es un foco fragmentado?»* Cuando una tarjeta tiene varios elementos enfocables y el lector da varios stops para una sola cosa; se evita con un contenedor `accessible`/`Pressable` con etiqueta completa.
- *«¿Cómo verificaron que cumple contraste?»* Con la fórmula WCAG implementada en `scripts/validar-paleta.mjs` sobre 18 pares, claro y oscuro.
- *«¿Por qué `Pressable` y no `TouchableOpacity`?»* Es el componente vigente, con estado `pressed`, `hitSlop` y accesibilidad completa.

---

15. Rendimiento

- **Listas:** se usa `ScrollView`, **no** `FlatList`. Justificación documentada: las listas son **acotadas** (registros de **un mes**, **4 áreas**, pocos vínculos); `FlatList` (virtualización) aporta cuando hay **cientos de elementos**. Queda en el *roadmap* para historiales largos (v1.2). Si te preguntan: *«con listas pequeñas la virtualización añade complejidad sin beneficio medible; si crecen, migramos a `FlatList`.»*
- **`useMemo`** para cálculos derivados (en `progreso.tsx`: rangos → buckets → insights → titular, encadenados con dependencias exactas). Evita recalcular en cada render.
- **React Compiler** (`reactCompiler: true` en `app.json`): memoiza automáticamente componentes y valores; reduce re-renders sin escribir `useMemo`/`memo` a mano.

- **Selectores específicos** (`useAppSelector((s) => s.settings.colorScheme)`) para que un componente solo se re-renderice cuando **su** dato cambia.
- **Consultas acotadas:** solo se piden los **rangos necesarios** (mes, o período actual + anterior) y solo las columnas necesarias (`select('id, symptom_type, …')`). `Promise.all` para consultas **paralelas** (síntomas + sesiones). Índices en Postgres.
- **Caché por mes** en `symptoms-slice` evita volver a pedir un mes ya cargado.
- **Arranque:** el *splash* se mantiene hasta tener fuentes, sesión y preferencias listas (evita parpadeos).
- **Sin animaciones pesadas ni destellos** (accesibilidad + rendimiento); «Reducir animaciones» disponible.

---

16. Decisiones documentadas (D-001 a D-020) para memorizar

Están en `docs/DECISIONES.md`. **El profesor puede preguntar por «decisiones documentadas en las Fases 1, 2 y 3».** Aprende el «qué» y el «por qué»:

ID	Decisión	Por qué (una línea)	Alternativa descartada
D-001	Expo + React Native + **Expo Router**	Es el entorno y el andamiaje; rutas por archivos	—
D-002	**Supabase** (Auth + Postgres + RLS)	Sync real entre dispositivos y permisos por fila para «acceso con autorización»	Firebase; solo local
D-003	MVP = §IV-C; médico y sensores fuera	Alcance defendible en el semestre	Hacer todo
D-004	**Pruebas con jest-expo** de lógica pura	Verificación fuerte; regla de «terminado»	Sin pruebas
D-005	**Redux Toolkit**	Lo pide el enunciado; asincronía, trazabilidad, reseteo al cerrar sesión	Context API
D-006	Documento final en **Markdown** dentro del repo	Versiona con el código	PDF directo
D-007	**Co-Authored-By** + reparto de módulos	Autoría equitativa y defendible	Falsear `--author`
D-008	Uso de IA libre, sin declaración	Duda resuelta por el profesor	—
D-009	Memoria **falla rápido**; háptica (`expo-haptics`)	Resaltado y veredicto no deben contradecirse; público con fatiga cognitiva	Evaluar al final
D-010	Vinculación por **código de 6 caracteres**; nombre/correo copiados en la fila; sin «Mis notas»	Sin cámara/deep linking; RLS impide leer `auth.users`	QR; tabla `profiles`
D-011	Pantalla de **Accesibilidad con controles reales**	Un control que no hace nada es peor que no tenerlo	«Perfiles por diagnóstico» (overlay widget)
D-012	**Paleta oscura real** de Stitch + escala de elevación	La derivada a mano fue rechazada tras probarla	Derivar de `inverse-*`
D-013	**Dictado con el teclado del sistema** + **lectura con `expo-speech`**; «el interruptor manda»	SDK 58 no trae reconocimiento de voz; privacidad; sin conexión	Reconocedor propio

ID	Decisión	Por qué (una línea)	Alternativa descartada
D-014	**Página web en Vercel** con `token_hash` para los correos	PKCE y Expo Go rompían el enlace a la app	Enlace profundo; código de 6 dígitos (plan B)
D-015	**Texto legal + constancia de consentimiento**	Datos de salud sensibles (Ley 1581)	Tabla de consentimientos (futuro)
D-016	**Perfil del médico**: solo lectura, permisos, caducidad, tabla propia	No convertir la app en canal clínico; no dar poderes de escritura	Chat; alertas; verificar RETHUS
D-017	Botón principal en oscuro: usar **`primary`**	Contraste 1,1:1 → 6,7:1 sin tocar la paleta	Cambiar `primaryContainer`
D-018	Actividad **Lógica** (series numéricas)	La propuesta la promete; «Próximamente» prometía algo irreal	Dejarla bloqueada
D-019	**`patient_name`** por trigger, solo con vínculo aceptado	El cuidador no puede leer `auth.users`	Join desde el cliente; `profiles`
D-020	Video con grabación real del emulador	Datos reales, reproducible	Solo ffmpeg con rótulos

> **Aviso sobre D-020:** el video lo hizo el equipo para mostrar a los compañeros y **ya no está en el repositorio** (no es un entregable); la decisión sigue en `DECISIONES.md` como historial.

> **Detalle fino de D-019:** el texto de D-019 menciona la política `caregiver_links_select_unclaimed_by_code`, pero esa política **se eliminó** en la migración `20261114` (hallazgo de seguridad). Hoy se reclama **solo** con `claim_invite`. Si te preguntan, lo correcto es: *«esa política se eliminó; el nombre solo se copia al aceptar»*.

---

17. Calidad del repositorio, Git y reparto del equipo

17.1 Lo que exige el enunciado (30 % «Funcionalidad y Repositorio»)

- App **funcional de extremo a extremo**, sin *mocks* injustificados ni cierres inesperados.
- Estructura limpia, código **tipado y modularizado**, **cero credenciales en el historial**.
- **Historial de commits que evidencie participación continua y equitativa** de los tres.
- **README completo**: instalación, variables de entorno, arquitectura, guía de ejecución y resumen de accesibilidad.

17.2 Cómo se cumple

Criterio	Evidencia
Tipado	TypeScript **estricto**; `npm run typecheck` (`tsc --noEmit`) limpio
Lint	`npx expo lint` (ESLint flat config con `eslint-config-expo`) limpio
Pruebas	`npm test` → 124 en verde
Sin secretos	`.env` en `.gitignore` y **nunca commiteado** (`git log -- .env` vacío); solo `.env.example`; solo se usa la clave **publishable** (nunca `service_role`)
Modular	`src/features/<módulo>/` con `api/logic/types/constants`
README	`README.md`: qué hace, stack, requisitos, instalación, variables, **10 migraciones en orden**, autenticación, ejecución, arquitectura, privacidad, accesibilidad
Estructura	Rutas en `src/app/`, lo demás fuera (regla del proyecto)

17.3 Git y co-autoría (D-007)

- **Autor** de los commits: Samuel (`SamuelBlanco-65`). **Amelyn** (`Amelynhs`) y **Mariana** (`MarianaTB-03`) figuran como **`Co-Authored-By`** en cada commit (verificado en GitHub: los tres avatares enlazan).
- **No se falseó `--author`:** decisión consciente (D-007) porque afirmaría una autoría **que no se puede defender** en el 40 %.
- **Realidad que debes conocer:** el historial muestra ~26 commits de Samuel y 1 de otro correo de Samuel; **Amelyn y Mariana no tienen commits propios** (solo coautoría). El requisito **RT-007 sigue «Pendiente»**. Para equilibrarlo, lo honesto es que **cada una haga al menos un commit propio** desde su equipo (acepten la invitación al repo y hagan un cambio real en su módulo).
- Formato del trailer:

```
Co-Authored-By: Amelynhs <258035243+Amelynhs@users.noreply.github.com>
Co-Authored-By: MarianaTB-03 <257896164+MarianaTB-03@users.noreply.github.com>
```

17.4 Qué debe dominar cada integrante (checklist por persona)

Samuel — Síntomas, datos, autenticación

- `lib/supabase.ts` línea por línea (`LargeSecureStore`, polyfills, `flowType: 'pkce'`).
- `features/auth/api.ts` (`toAuthError`, `signUp`/`signIn`/`toSession`), `auth-slice.ts` (estados, `openAuthLink` con `condition`).
- `registrar-sintoma/*`, `calendario.tsx`, `symptoms-slice.ts`, `symptoms/api.ts`, `symptoms/calendar.ts`.
- Migración `symptom_logs.sql` y qué hace cada política RLS; `delete_my_account`.
- `web-auth/` y por qué existe (D-014); `legal/` y consentimiento (D-015).
- Contraste y daltonismo (`scripts/validar-paleta.mjs`) si fue quien lo ejecutó.

Amelyn — Cuidador, recordatorios, apoyo

- Máquina de estados del vínculo; `claim_invite`; `caregiver-slice.ts` (lado paciente vs. lado cuidador).
- `permisos.tsx`, `vincularme.tsx`, `(cuidador)/*`, `nueva-observacion.tsx`.
- RLS de `caregiver_observations` (el paciente **no** la ve), `reminders` y su trigger, `shared_symptom_logs`.
- Por qué el código de invitación del cuidador tiene la limitación (oráculo, no atómico) y cómo se arregló en el médico.
- `support/` (contactos de crisis, transcripción en lugar de media).

Mariana — Actividades y progreso

- `features/activities/logic.ts` completo (generadores, `nextLevel`, Lógica con `rng` inyectable).
- `activities-slice.ts`: máquina de estados `phase`, `sessionStarted`, `memoriaShapeTapped` (falla rápido), `roundContinued`, `condition` de `fetchAreaProgress`.
- `actividad/[area].tsx` (háptica, `useReduceMotion`, `persistAreaLevel`, `recordActivitySession`).
- `progress/logic.ts` y `progreso.tsx` (rangos, buckets, insights, gráfico accesible).
- Migraciones `activity_progress`, `activity_sessions`, `activity_area_logica`.

Todos (base común): Redux Toolkit (sección 8), navegación (7), accesibilidad (14), `ThemedText`, `ScreenState`, RLS (10), decisiones (16).

---

18. Errores reales que encontramos y cómo se arreglaron

Es una de las mejores cosas que puedes contar: demuestran que **probamos en dispositivo** y que entendemos las causas. Memoriza **causa → arreglo → archivo**.

#	Error (síntoma)	Causa raíz	Arreglo	Dónde
1	Un registro sin confirmar **parecía autenticado** y se perdía al reiniciar	Se armaba la sesión con `data.user` en vez de `data.session`	Usar `data.session`	`features/auth/api.ts`
2	La sesión **no sobrevivía a un reinicio**	`expo-secure-store` limita ~2 KB y el JWT lo supera; fallaba en silencio	`LargeSecureStore` (AES-256 + clave en SecureStore, blob en AsyncStorage)	`lib/supabase.ts`
3	**Atención salía en blanco**	La fase inicial era `'memorizar'` para todas las áreas salvo Movimiento; ese bloque solo existe en Memoria	Fase inicial `'memorizar'` solo si `area === 'memoria'`	`activities-slice.ts`
4	Errores de Supabase siempre caían en «Algo salió mal»	`error instanceof PostgrestError` nunca era verdadero	Detección por **forma** (`code`/`message`)	`*/api.ts`
5	En «Crear cuenta» un fallo (correo duplicado, rate limit) **no mostraba nada**	El error solo se renderizaba en modo `login`	Mostrarlo en ambos modos	`acceder.tsx`
6	Las pestañas se veían en gris «Material You»	`NativeTabs` no heredaba el tema	Pasar `backgroundColor`, `iconColor`, `labelStyle`, `indicatorColor`	`(paciente)/_layout.tsx` y `(cuidador)/_layout.tsx`
7	Banners de error **ilegibles en oscuro**	Roles `tertiaryFixed` (fijos) con texto de roles que **sí** cambian por esquema	Usar `onTertiaryFixed/onTertiaryFixedVariant`	`text-field.tsx`, `calendario.tsx`, `paso-3.tsx`…
8	Barras del gráfico de Progreso **no seleccionables con TalkBack**	Eran decorativas	`Pressable` con rol, etiqueta y estado + `accessibilityLiveRegion`	`progreso.tsx`
9	Actividades mostraba **«No hay sesión activa»** al cambiar de cuenta	La pestaña montada lanzaba `fetchAreaProgress` sin sesión tras `resetAll`	`condition` en el thunk	`activities-slice.ts`

#	Error (síntoma)	Causa raíz	Arreglo	Dónde
10	Texto **recortado** con fuente grande	`lineHeight` fijo no escalaba	Escalar `lineHeight` a mano	`themed-text.tsx`
11	Botones **ilegibles** en oscuro (≈ 1,1:1)	`onPrimary` sobre `primaryContainer`	`colors.primary` (D-017)	~25 archivos
12	`share_sintomas` del cuidador **no tenía efecto**	Ninguna política lo aplicaba	`shared_symptom_logs`	migración `20261115`
13	Las actividades «se contradecían» (figura resaltada vs. resultado)	Se evaluaba solo al final	**Falla rápido**	`activities-slice.ts`
14	«Paso 1 de 1» no se entendía en Memoria	La misma palabra «Paso» para dos cosas	`unit` en `SessionHeader`: «Figura N de M» / «Paso N de M» / «Ronda N de 3»	`session-header.tsx`
15	Lint rechaza `setState` en un `useEffect` (resaltado transitorio y fetch)	Regla `react-hooks/set-state-in-effect`	`setState` en el **manejador** o el fetch en un **thunk**	`actividad/[area].tsx`, `acompanar.tsx`
16	Un enlace de correo daba **ventana negra**	PKCE + Expo Go sin esquema + Gmail	Página web en Vercel (D-014)	`web-auth/`
17	Un Metro viejo producía errores falsos (`Cannot read property 'status'`)	Dos bundles a la vez	Cerrar el proceso viejo	(entorno)

19. Limitaciones conocidas y trabajo futuro

Preséntalas tú (con su razón) antes de que te las señalen.

Limitación	Por qué	Mejora (versión)
Solo Android verificado (sin iOS)	Entorno del equipo (Windows)	Verificar en iOS
Sin caché offline ni cola de sincronización	Alcance del semestre; se muestra error claro	v1.2: caché + cola
Recordatorios **sin notificaciones del sistema** (solo alerta visual y háptica **dentro** de la app)	Notificaciones locales añaden permisos y dependencias	v1.2
Código de invitación del cuidador generado por el cliente y límite **no atómico**	Se endureció primero el médico; el del cuidador es de migraciones anteriores	v1.1: replicar `create_doctor_invite` + `pg_advisory_xact_lock`
TalkBack **solo en el perfil paciente**; sin fuente ampliada ni iOS	Tiempo	v1.1

Limitación	Por qué	Mejora (versión)
`ScrollView` en vez de `FlatList`	Listas pequeñas	v1.2
Sin pruebas de UI ni de integración	Criterio D-004 (solo lógica pura)	—
Consentimiento en metadatos del cliente	Alcance	Tabla de consentimientos con trigger
Responsable de datos sin domicilio/teléfono	Pendiente legal	Completar (Decreto 1377, art. 13)
Nombre del paciente «congelado» al aceptar (D-019) y nombre/correo del cuidador al solicitar (D-010)	Sin tabla `profiles`	Tabla `profiles`
Ícono de la app es el de Expo	Tiempo	Identidad propia
No verificado profesionalmente el médico (RETHUS autodeclarado)	Fuera de alcance	v2.0
Hoja de ruta: v1.1 (código servidor + TalkBack completo) · v1.2 (offline, notificaciones, `FlatList`) · v2.0 (versión auditiva completa, verificación de profesionales, publicar en Google Play con EAS)	—	—

20. Banco de preguntas con respuestas modelo

> **Cómo estudiar esto:** tapa la respuesta, contesta en voz alta con la fórmula de la sección 0.3 y compara. Las respuestas son **modelo**: adapta las palabras, mantén los datos.

A. Visión y producto

1. ¿Qué es Neurotrack EM y para quién? App móvil de seguimiento, rehabilitación y acompañamiento para personas con esclerosis múltiple, su cuidador y, opcionalmente, su médico. Complementa —no reemplaza— el seguimiento médico entre consultas; no diagnostica ni usa sensores.

2. ¿Qué problema resuelve? La EM es muy variable y los controles médicos no permiten monitoreo continuo; entre consultas, paciente y cuidador quedan sin acompañamiento, y el cuidador nota cambios que no sabe comunicar. La app da un historial útil y un espacio de apoyo.

3. ¿Qué la diferencia de la competencia? Integra tres perfiles con **permisos que controla el paciente y se aplican en la base de datos**, da un **rol activo al cuidador**, no depende de sensores, está en **español y contexto colombiano**, y trata la accesibilidad como requisito del dominio. Frente a My MS Manager (solo registro), Floodlight (clínica, sensores), MSCopilot (mucho texto) y MyTherapy (genérico).

4. ¿Por qué no usaron sensores o biomarcadores? Requieren validación clínica, financiamiento y años de estudio; la propuesta (§IV-C) los dejó fuera del semestre. Además (RT-003) verificamos que no hay imports de sensores.

5. ¿La app diagnostica? No (RT-004). Hay avisos fijos «no diagnostica», el médico ve **tendencias neutras sin alertas** y nunca «mejor/peor».

B. Arquitectura y React Native

6. Explíquenme la arquitectura. Cliente Expo/React Native con Expo Router; pantallas que solo leen del store y despachan acciones; Redux Toolkit con 9 slices y `createAsyncThunk`; cada módulo en

`features/` con `api.ts` (único que habla con Supabase) y `logic.ts` (reglas puras con pruebas); Supabase como backend (Auth + Postgres con RLS + funciones). Un sitio estático en Vercel para los correos.

7. ¿Por qué separan `api.ts`, `logic.ts` y las pantallas? Para que la UI no dependa de la red, para poder **probar** la lógica sin emulador y para cambiar el backend tocando un solo archivo.

8. ¿Cómo funciona la navegación? Expo Router por archivos. Grupos `(paciente)`, `(cuidador)`, `(medico)`, `(auth)`; los dos primeros con `NativeTabs`, el médico con pila; el asistente de síntomas es un modal. `index.tsx` redirige según sesión y perfil con `Redirect` y el diccionario `PROFILE_HOME`.

9. ¿Qué es `_layout.tsx`? El archivo que define **cómo se navegan sus rutas hijas** (pila, pestañas, modal) y donde se montan proveedores. El raíz monta el `StoreProvider` y mantiene el splash hasta que haya fuentes, sesión y preferencias.

10. ¿Qué es un grupo de rutas `(paciente)`? Una carpeta entre paréntesis organiza rutas sin añadir segmento a la URL; permite un `_layout` propio por perfil.

11. ¿Qué hacen `useEffect`, `useMemo`, `useRef` aquí? `useEffect` dispara cargas (`dispatch(fetchMonthLogs(...))`) cuando cambian sus dependencias; `useMemo` memoiza cálculos derivados (Progreso); `useRef` guarda referencias a `TextInput` para el manejo de foco y a temporizadores.

12. ¿Qué es el React Compiler que activaron? Un compilador que memoiza automáticamente componentes y valores para evitar re-renders innecesarios; se activa con `experiments.reactCompiler` en `app.json`.

13. ¿Por qué `ScrollView` y no `FlatList`? Las listas son acotadas (un mes, cuatro áreas, pocos vínculos). `FlatList` virtualiza y sirve con cientos de ítems; queda en el roadmap para historiales largos.

14. ¿Qué es `StyleSheet.create` y por qué lo usan? Define estilos con identificadores optimizados y validados; centraliza el estilo del componente. Los valores (espaciado, radios, colores) salen de **tokens** del tema, no de números sueltos.

15. ¿Cómo manejan los tokens de diseño? `src/theme/`: colores M3 (claro, oscuro, alto contraste), tipografía (`TypeScale`) y `Spacing/Radius/MinTouchSize`. Se consumen con `useThemeColors()`.

C. Manejo de estado

16. ¿Qué usaron para el estado: Redux o qué? Redux Toolkit con `react-redux`. Store en `src/store/store.ts`, 9 slices, hooks tipados `useAppDispatch`/`useAppSelector`.

17. ¿Por qué Redux Toolkit y no Context? El enunciado lo permite; hay estado compartido y asíncrono (sesión, vínculos, borradores, caché por mes) y reseteo global; Context no trae asincronía ni trazabilidad y re-renderiza a todos los consumidores (D-005).

18. ¿Qué es un slice? ¿Un reducer? ¿Una acción? ¿Un thunk? Slice = trozo de estado con sus reducers y acciones (`createSlice`); reducer = función `(estado, acción) → estado`; acción = `{type, payload}`; thunk = función asíncrona despachable (`createAsyncThunk`) que genera `pending/fulfilled/rejected`.

19. ¿Cómo se hace una llamada asíncrona? Con un `createAsyncThunk`: lee `getState`, llama a la API del módulo y devuelve el resultado o `rejectWithValue(error)`; los `extraReducers` actualizan `status` (`cargando/listo/error`) y los datos; la pantalla muestra `ScreenState` según ese status.

20. ¿Para qué sirve `rejectWithValue`? Para entregar al reducer `rejected` un error **tipado y ya traducido** en lugar del `Error` genérico.

21. ¿Qué es `condition` en un thunk? Una función que decide **si el thunk se ejecuta**. La usamos en `fetchAreaProgress` (solo con sesión) y en `openAuthLink` (no canjear dos veces un enlace de un solo uso).

22. ¿Qué es `requestId` y para qué lo usan? Cada ejecución de un thunk tiene un id único. En el resumen del médico se guarda el del último pedido y se **descartan** respuestas de pedidos anteriores (condición de carrera al cambiar rápido de período).

23. ¿Qué pasa con el estado al cerrar sesión? Se despacha `resetAll`; el reducer raíz reinicia todos los slices salvo `settings` y `auth`. Así el siguiente usuario del teléfono no ve datos de salud ajenos.

24. ¿Cómo persisten las preferencias? `store.subscribe` guarda `settings` en AsyncStorage cuando cambia; al abrir, `hydrateSettings` las carga. Sin cifrar porque no son datos de salud.

25. ¿Qué es Immer y por qué pueden «mutar» en los reducers? Redux Toolkit usa Immer: escribimos `state.x = y` y por debajo produce un estado nuevo inmutable.

26. ¿Dónde vive el borrador del asistente de síntomas y por qué? En `state.symptoms.draft` (Redux), porque lo comparten tres pantallas y debe sobrevivir a la navegación; no se guarda hasta el paso 3.

27. ¿Cómo evitan estados imposibles en una actividad? Máquina de estados con `phase` y cada reducer valida la fase (`if (session.phase !== 'responder') return`).

D. Datos, API y backend

28. ¿Qué backend usan y por qué? Supabase (Auth + Postgres + RLS), capa gratuita. Da sincronización real y permisos por fila; el enunciado prohíbe datos simulados. No hay servidor propio: las reglas viven en la base.

29. ¿Cómo consumen la API? Con `supabase-js`: `supabase.from('tabla').select/insert/upsert…` para CRUD y `supabase.rpc('funcion', params)` para funciones. Todo encapsulado en `features/<módulo>/api.ts`.

30. ¿Qué es RLS? Row Level Security: políticas de Postgres que filtran **filas** por usuario (`auth.uid()`). Sin políticas nadie accede. Garantiza que cada paciente solo vea lo suyo y que cuidador y médico vean solo lo autorizado.

31. ¿Por qué RLS y no validar en la app? La clave del cliente es pública y cualquiera podría llamar a la API sin la app. Una regla en la app se evita; una en la base no.

32. ¿Qué diferencia hay entre `using` y `with check`? `using` filtra qué filas existentes puede ver/tocar; `with check` valida qué valores puede escribir.

33. ¿Para qué las funciones `security definer`? RLS filtra filas, no columnas, y no deja leer datos ajenos. Una función `security definer` corre con los privilegios de su dueño y contiene la regla (p. ej. ocultar la nota, o reclamar un código); se restringe con `revoke … from public, anon` y `grant … to authenticated`.

34. ¿Cómo evitan que el cuidador lea las notas del paciente? Los terceros solo leen síntomas por `shared_symptom_logs`, que devuelve `note = null` salvo médico con `share_notas`. Además el cuidador no tiene política `select` directa sobre `symptom_logs`.

35. ¿Cómo funciona el consentimiento? Máquina de estados `pendiente → solicitado → aceptado/rechazado → revocado`. Solo el paciente puede pasar a `aceptado` (política `update_as_patient`), eligiendo permisos. Puede revocar siempre, con corte inmediato en el servidor.

36. ¿Qué es un trigger y cuáles tienen? Función que Postgres ejecuta automáticamente al insertar/actualizar. Tenemos tres: `reminders_guard_caregiver_update` (el cuidador solo marca «hecho»), `doctor_links_guard_patient_update` (protege columnas y valida transiciones) y `caregiver_links_set_patient_name` (rellena el nombre solo con vínculo aceptado).

37. ¿Cómo se guardan los datos de la sesión? Cifrados con AES-256: clave aleatoria en SecureStore y blob cifrado en AsyncStorage (`LargeSecureStore`), porque el JWT excede el límite de ~2 KB de SecureStore.

38. ¿Qué es la clave `publishable`? ¿Es un secreto? Es la clave pública del cliente (equivale a la antigua `anon`). No es secreto; la seguridad la dan las políticas RLS. Nunca usamos `service_role`.

39. ¿Cómo se manejan los errores de red? Cada `api.ts` convierte el error en una clase tipada (`sin-conexion` para `TypeError`, `servidor`, `validacion`, etc.) con mensaje en español; el slice la guarda y `ScreenState` la muestra.

40. ¿Hay modo offline? Se detecta el fallo y se muestra un error claro, pero no hay caché ni cola de sincronización (limitación declarada; roadmap v1.2).

41. ¿Cómo se borra una cuenta? RPC `delete_my_account` borra de `auth.users` y el `on delete cascade` elimina todo lo demás; luego `signOut({ scope: 'local' })`.

42. ¿Cómo evitan que adivinen códigos de invitación? Alfabeto de 32 caracteres sin ambiguos; límite de 10 fallos cada 15 min (`claim_attempts`); en el médico además código generado en servidor, caducidad de 7 días y candado atómico. En el cuidador el código lo genera el cliente y el límite no es atómico: limitación conocida (v1.1).

43. ¿Por qué copiaron el nombre del paciente en la fila del vínculo? El cuidador no puede leer `auth.users`; un trigger lo copia solo cuando el vínculo está aceptado y sobrescribe cualquier valor del cliente.

44. ¿Qué es PKCE y por qué lo usan? Flujo OAuth con código de un solo uso y un verificador guardado en el dispositivo; más seguro que tokens en la URL. Por eso el enlace de correo solo funciona en el dispositivo que lo pidió, y por eso usamos la página `web-auth/` con `token_hash`.

45. ¿Por qué una página web en Vercel? Porque el enlace a la app fallaba (PKCE, Expo Go sin esquema propio, Gmail no abre esquemas). La página llama a `verifyOtp` al pulsar el botón y funciona desde cualquier dispositivo (D-014).

E. Accesibilidad

46. ¿Cómo implementaron la accesibilidad? Props nativas en todos los controles (`accessibilityRole/Label/Hint/State`, `accessibilityLiveRegion`), agrupamiento con `accessible`/`Pressable`, áreas de 48 dp con `hitSlop`, `ThemedText` con `lineHeight` escalado, contraste validado por script, modo oscuro y alto contraste, háptica, dictado y lectura en voz alta; verificado con TalkBack en celular real.

47. ¿Por qué escalan `lineHeight` a mano? (Ver 14.6.)

48. ¿Qué hace `accessibilityRole/Label/Hint/State`? (Ver 14.2.)

49. ¿Cómo manejan el foco? (Ver 14.5.)

50. ¿Cómo garantizan el área táctil mínima? `MinTouchSize = 48` en `minHeight/minWidth` y `hitSlop` para íconos pequeños.

51. ¿Cómo verificaron el contraste y el daltonismo? `scripts/validar-paleta.mjs`: 18 pares con la fórmula WCAG y simulación de protanopia, deuteranopia y tritanopia con matrices de Machado.

52. ¿Cómo comunican estados sin depender del color? Barras + palabra (severidad), ✓/✕ + texto, flechas + texto, borde + ícono + texto en errores.

53. ¿Qué es un `accessibilityLiveRegion`? Hace que el lector anuncie cambios de contenido sin que el usuario los busque (`polite` o `assertive`).

54. ¿Cómo probaron con TalkBack? En un celular Android real: 14 capturas con el foco del lector (`artifacts/talkback/`). Encontramos que las barras del gráfico de Progreso no eran seleccionables y lo arreglamos.

55. ¿Qué pasa con TalkBack y la lectura en voz alta? Android da prioridad a TalkBack y corta la voz de la app: es correcto; por eso el interruptor está apagado por defecto.

56. ¿Por qué el dictado usa el teclado del sistema? SDK 58 no trae reconocimiento de voz y las librerías externas exigen build nativa (Expo Go no las admite); además el audio lo procesa el teclado, no la app.

57. ¿Cómo soportan baja visión? Texto escalable (app × sistema), Atkinson Hyperlegible, alto contraste, modo oscuro sin blanco puro, un elemento principal por pantalla.

F. Decisiones, proceso y equipo

58. ¿Qué decisión técnica defenderían más y cuál cambiarían? Defender: la seguridad en la base (RLS + funciones), porque no se puede evadir. Cambiar: el código de invitación del cuidador (pasarlo al servidor con límite atómico, como el del médico).

59. ¿Qué fue lo más difícil? La sesión segura (límite de SecureStore), el consentimiento con RLS sin filtrar notas, el contraste en modo oscuro y los correos que funcionaran en cualquier dispositivo.

60. ¿Cómo se repartieron el trabajo? Samuel: síntomas, datos y autenticación; Amelyn: cuidador, recordatorios y apoyo; Mariana: actividades y progreso. El médico, voz y accesibilidad fueron extensiones del equipo. Commits con `Co-Authored-By` de los tres.

61. ¿Usaron IA? El uso era libre (D-008); lo importante es que cada quien entienda y defienda su código. (Responde con honestidad y demuestra dominio.)

62. ¿Cómo verificaron que todo funciona? `tsc`, `eslint`, 124 pruebas, pruebas manuales en emulador y celular, capturas en `artifacts/`.

63. ¿Qué no probaron? UI automatizada, integración con Supabase, iOS, fuente ampliada ni TalkBack en cuidador/médico.

64. ¿Qué agregarían con más tiempo? Offline, notificaciones locales, código del cuidador en servidor, `FlatList`, iOS, identidad visual propia y verificación de profesionales.

G. Trampas y contrapreguntas frecuentes

65. «¿Por qué `useAppSelector` y no `useSelector`?» Es el mismo hook con el tipo del `RootState` ya aplicado: autocompleta y evita errores.

66. «¿Qué pasa si el usuario toca dos veces "Guardar"?» El thunk pone `submitStatus = 'guardando'`; en `paso-3.tsx` eso se pasa a `WizardActions` como `primaryLoading`, que deshabilita el botón (`disabled={primaryDisabled || primaryLoading}`), muestra «Guardando…» y marca `accessibilityState={{ busy }}`.

67. «¿Qué pasa si falla la red al guardar el nivel de una actividad?» No se bloquea: el nivel ya vive en la sesión local; `persistAreaLevel.rejected` marca `saveStatus='error'` (RNF-012).

68. «¿Dónde está la lógica de subir de nivel?» `nextLevel` en `features/activities/logic.ts`, con `ROUNDS_TO_LEVEL_UP = 2` en `constants.ts`; Movimiento sube por completar la sesión (`movimientoSessionFinished`).

69. «¿Cómo sabe el calendario qué color/barra poner a cada día?» `buildMonthGrid` calcula la **peor severidad** del día con `SEVERITY_RANK`; la UI la dibuja como barras (1/2/3) + leyenda.

70. «¿Por qué el perfil se elige al abrir y no va en la cuenta?» Decisión de alcance: una cuenta puede usarse como paciente, cuidador o médico (hay vínculos entre cuentas); queda como consecuencia documentada en D-016.

H. Backend (ver sección 10.12)

71. «Supabase es la base de datos; ¿dónde está el backend?» Supabase **es** el backend: Auth (JWT), una API REST generada (PostgREST), funciones SQL y triggers que corren en el servidor, sobre Postgres. Lo que no hay es un servidor escrito por nosotros. El código de acceso está en `src/features/*/api.ts` y `src/lib/supabase.ts`; la lógica y la seguridad del servidor, en `supabase/migrations/`.

72. «Sin un backend intermediario, ¿no queda expuesta la base?» No: la app no se conecta directo a Postgres, sino a la API de Supabase, que valida el JWT y aplica RLS en cada consulta. La clave del cliente es pública por diseño; la `service_role`, la única que se salta RLS, nunca está en la app. Riesgo real que reconocemos: un error en una política expone datos (nos pasó y la auditoría lo corrigió); un intermediario sería una segunda barrera.

73. «¿Dónde están el router, los controladores, los servicios y la API REST?» La API REST y su enrutamiento los genera PostgREST; las funciones RPC hacen de controladores; la autorización es RLS; las reglas sensibles están en SQL y las no sensibles en `logic.ts`; `api.ts` es el repositorio y `types.ts` los modelos. El «router» del código es Expo Router, que es de **frontend** (pantallas), no de API.

74. «¿Habría sido mejor tener un backend propio?» Para este alcance, no: sería sobreingeniería sin secretos ni tiempo real, y añadiría superficie de ataque y mantenimiento. Sí sería mejor práctica mover al servidor el código de invitación del cuidador, la validación de niveles, la constancia de consentimiento y la auditoría, con funciones SQL o Edge Functions; en producción lo haríamos. Un proyecto como EA-FC, con reglas de competencia, tiempo real y clave secreta, sí lo justifica (por eso allí usamos FastAPI).

75. «¿Qué lógica de negocio corre en el cliente y por qué no es un problema?» La generación de rondas y la subida de nivel de las actividades, los cálculos de Progreso y del calendario, y las comparaciones del médico, todas sobre **datos que el usuario ya puede leer**. Lo peor que puede pasar es que un usuario falsee **su propio** nivel, sin impacto en otros ni dato clínico. Todo lo que afecta a terceros o a permisos está en el servidor.

---

21. Inspección de código en vivo: ejercicios para practicar

El profesor puede pedir **«ubica X»**, **«explica línea por línea»** o **«haz este cambio»**. Practica con tu editor.

21.1 «Ubica…» (tiempo objetivo: < 20 segundos cada una)

Pídele a un compañero…	Respuesta
¿Dónde se crea el store?	`src/store/store.ts`
¿Dónde se reinicia el estado al cerrar sesión?	`src/store/actions.ts` (`resetAll`) y `store.ts` (reducer raíz)
¿Dónde se guarda la sesión de forma segura?	`src/lib/supabase.ts` → `LargeSecureStore`
¿Dónde se escala el `lineHeight`?	`src/components/themed-text.tsx`
¿Dónde está el mínimo táctil?	`src/theme/spacing.ts` → `MinTouchSize = 48`
¿Dónde se valida el contraste?	`scripts/validar-paleta.mjs`
¿Dónde se decide a qué pantalla ir al abrir?	`src/app/index.tsx` + `src/features/profile/constants.ts`
¿Dónde se define la regla de subir de nivel?	`src/features/activities/logic.ts` (`nextLevel`) y `constants.ts`
¿Dónde están las políticas RLS de síntomas?	`supabase/migrations/20261108000000_symptom_logs.sql`
¿Dónde está la función que reclama un código?	`20261114…harden_links_and_account.sql` (`claim_invite`)
¿Dónde se oculta la nota al cuidador?	`20261115…doctor_links.sql` (`shared_symptom_logs`)
¿Dónde se mueve el foco al campo con error?	`src/app/(auth)/acceder.tsx` (`validar()`)
¿Dónde se elige la voz en español?	`src/features/audio/voice.ts` (`chooseSpanishVoice`)
¿Dónde se guardan las preferencias de accesibilidad?	`features/settings/persistence.ts` + `settings-slice.ts`
¿Dónde está la pantalla de actividades en curso?	`src/app/actividad/[area].tsx`
¿Dónde están los textos de las guías de Apoyo?	`src/features/support/constants.ts`
¿Dónde se calcula la comparación del médico?	`src/features/doctor/logic.ts` (`compareSymptoms`)
¿Dónde se define la tipografía?	`src/theme/typography.ts` (cargada en `_layout.tsx`)

21.2 «Hagan este cambio» (practica y recuerda qué más tocar)

1. **Subir de nivel con 3 de 3 rondas.** `ROUNDS_TO_LEVEL_UP = 2` → `3` en `features/activities/constants.ts`. *(La prueba de `nextLevel` usa la constante; si hay pruebas con valores literales, ajústalas.)*
2. **Que las sesiones tengan 4 rondas.** `ROUNDS_PER_SESSION = 3` → `4` en `constants.ts`; revisa los textos «Ronda N de 3» de `[area].tsx` (`getHeaderProgress`).
3. **Agregar un tipo de síntoma («Temblor»).** Hay que tocar **tres sitios**: `types.ts` (`SymptomType`), `constants.ts` (`SYMPTOM_TYPE_OPTIONS` e íconos) **y la base de datos** (el `check (symptom_type in (...))` de `symptom_logs`, con una migración nueva). *Es una trampa clásica: si solo cambias la app, el `insert` falla en Postgres.*
4. **Cambiar el tamaño del texto «grande».** `TEXT_SCALE_MULTIPLIER` en `settings-slice.ts` (`grande: 1.15`).
5. **Cambiar el largo máximo de la nota.** `NOTE_MAX_LENGTH = 300` en `paso-3.tsx` (y el contador usa esa constante).
6. **Aumentar el mínimo táctil.** `MinTouchSize` en `spacing.ts`: se propaga a todos los componentes que lo usan.

7. **Cambiar la velocidad de la voz.** `rate: 0.9` en `features/audio/speech.ts`.
8. **Añadir una pestaña nueva al paciente.** Crear `src/app/(paciente)/nueva.tsx` y un `NativeTabs.Trigger name="nueva"` en `(paciente)/_layout.tsx`.
9. **Añadir una preferencia (p. ej. «animaciones lentas»).** `SettingsState` + `DEFAULT_SETTINGS` + reducer en `settings-slice.ts`; el control en `accesibilidad.tsx`; se persiste solo (el `subscribe` de `store.ts` guarda todo `settings`).
10. **Nueva política RLS.** Escribir una migración **idempotente** (`drop policy if exists` + `create policy`) y ejecutarla en el SQL Editor.
11. **Cambiar el texto de un error.** Está en el `toXError` del `api.ts` del módulo.
12. **Cambiar cuánto dura la invitación del médico.** `invite_expires_at … default now() + interval '7 days'` en la migración (y la función `create_doctor_invite`).

21.3 «Explica línea por línea» — qué fragmentos prepararte

1. `LargeSecureStore` (sección 9.2).
2. `store.ts` (sección 8.2).
3. Un thunk completo + sus `extraReducers` (`fetchMonthLogs`, sección 8.5).
4. `ThemedText` (sección 14.6).
5. `ProfileOption` (sección 14.3).
6. `nextLevel` y `generateMemoriaRound` (sección 12.3).
7. `claim_invite` y `shared_symptom_logs` (sección 10.5).
8. `validar()` de `acceder.tsx` (sección 14.5).
9. `speak()` y `chooseSpanishVoice` (sección 13.8).
10. `_layout.tsx` raíz (sección 7.3).

---

22. Glosario

Término	Significado
EM	Esclerosis múltiple: enfermedad neurodegenerativa del sistema nervioso central
Neuritis óptica	Inflamación del nervio óptico; frecuente en EM; deja secuelas en contraste, color y fotofobia
UVP	Propuesta única de valor
MVP	Producto mínimo viable
Expo / Expo Go	Plataforma sobre React Native / app para ejecutar proyectos sin compilar nativo
Expo Router	Navegación por archivos
Redux Toolkit (RTK)	Librería oficial para escribir Redux de forma simple
Store / Slice / Reducer / Action / Thunk	Ver sección 8.1
Immer	Permite «mutar» estado de forma segura dentro de reducers
Selector	Función que lee una parte del estado
Supabase	Backend como servicio sobre PostgreSQL
JWT	Token firmado que identifica la sesión; de él sale `auth.uid()`
RLS	Row Level Security: permisos por fila en Postgres
RPC	Llamada a una función de la base (`supabase.rpc`)

Término	Significado
`security definer`	Función que se ejecuta con los privilegios de su dueño
Trigger	Función automática ante un cambio en una tabla
Migración	Archivo SQL versionado que cambia el esquema
Idempotente	Se puede ejecutar varias veces con el mismo resultado
PKCE	Flujo OAuth con código de un solo uso y verificador local
`token_hash` / `verifyOtp`	Mecanismo para validar enlaces de correo desde cualquier dispositivo
SecureStore / AsyncStorage	Almacenamiento seguro (pequeño) / almacenamiento general (sin cifrar)
AES-256 (CTR)	Cifrado simétrico con clave de 256 bits
WCAG 2.2 AA	Pautas de accesibilidad nivel AA
POUR	Perceptible, Operable, Comprensible, Robusto
TalkBack / VoiceOver	Lectores de pantalla de Android / iOS
`accessibilityRole/Label/Hint/State`	Props que describen el elemento al lector
`accessibilityLiveRegion`	Anuncia cambios dinámicos
Foco fragmentado	Un solo control con varios stops de lectura innecesarios
`hitSlop`	Margen táctil extra alrededor de un elemento
dp / pt	Unidades de densidad independiente (Android / iOS)
`lineHeight`	Altura de línea del texto
Contraste (razón)	`(L1+0,05)/(L2+0,05)`; 4,5:1 texto, 3:1 componentes
Protanopia / Deuteranopia / Tritanopia	Tipos de daltonismo (rojo, verde, azul)
ΔE (CIE76)	Diferencia de color perceptual
Material 3 (M3)	Sistema de diseño con «roles» de color
Háptica	Respuesta táctil (vibración)
Oráculo (de códigos)	Pista que revela si un código existe, permitiendo adivinarlo
Condición de carrera	Dos operaciones asíncronas que terminan en orden inesperado
Mock	Dato o servicio simulado (el enunciado los prohíbe sin justificación)

23. Mapa completo de archivos

23.1 Pantallas (`src/app/`)

Archivo	Qué es
`_layout.tsx`	Raíz: `StoreProvider`, arranque de sesión/preferencias, splash, `Stack`
`index.tsx`	Selección de perfil + redirecciones
`(auth)/_layout.tsx`, `acceder.tsx`, `recuperar.tsx`	Iniciar sesión / crear cuenta / recuperar contraseña
`(paciente)/_layout.tsx`	Pestañas del paciente
`(paciente)/inicio.tsx`	Inicio: ánimo del cuidador, «Tu plan de hoy», recordatorios, accesos
`(paciente)/calendario.tsx`	Calendario del mes + detalle del día

Archivo	Qué es
(paciente)/actividades.tsx	Sesión de hoy + 4 áreas
(paciente)/progreso.tsx	Progreso: período, gráfico/tabla, insights
(paciente)/apoyo.tsx	Apoyo: crisis + guías
(cuidador)/_layout.tsx	Pestañas del cuidador
(cuidador)/inicio.tsx, acompanar.tsx, observaciones.tsx, recordatorios.tsx, apoyo.tsx	Módulo del cuidador
(medico)/_layout.tsx, pacientes.tsx, vincular-paciente.tsx, paciente/[id].tsx	Perfil del médico
registrar-sintoma/_layout.tsx, paso-1/2/3.tsx	Asistente de síntomas (modal)
actividad/_layout.tsx, actividad/[area].tsx	Sesión de actividad
apoyo/_layout.tsx, apoyo/[id].tsx	Detalle de guía
legal/[doc].tsx	Términos y privacidad
cuenta.tsx	Mi cuenta (cerrar sesión, borrar cuenta)
accesibilidad.tsx	Preferencias de accesibilidad
acerca.tsx	«¿Qué es Neurotrack EM?»
mi-medico.tsx	El paciente gestiona su médico
permisos.tsx	El paciente gestiona sus cuidadores
vincularme.tsx	El cuidador escribe el código
nueva-observacion.tsx	Crear observación
correo/confirmar.tsx, correo/restablecer.tsx	Respaldo del flujo anterior (no usado)

23.2 Componentes (`src/components/`)

`themed-text.tsx` (texto con tema y `lineHeight` escalado) · `themed-view.tsx` (contenedor con fondo del tema) · `screen-state.tsx` (carga/error/vacío) · `text-field.tsx` (campo con error doble-codificado) · `profile-option.tsx` (tarjeta radio) · `account-button.tsx` (botón de cuenta).

23.3 Módulos (`src/features/`)

Carpeta	Archivos principales
auth	api.ts, types.ts, validation.ts, deep-link.ts, recover-password-form.tsx
symptoms	api.ts, types.ts, constants.ts, calendar.ts, summary.ts, wizard-header.tsx, wizard-actions.tsx
activities	api.ts, logic.ts, types.ts, constants.ts, area-tile.tsx, shape-button.tsx, feedback-badge.tsx, session-header.tsx
progress	logic.ts, types.ts, constants.ts
caregiver	api.ts, types.ts
reminders	api.ts, types.ts, constants.ts
doctor	api.ts, logic.ts, types.ts, constants.ts, components.tsx, severity-bars.tsx
support	constants.ts, types.ts, crisis-section.tsx, guide-list.tsx
audio	speech.ts, voice.ts, scripts.ts, screen-scripts.ts, speak-button.tsx, dictation-button.tsx, use-activity-narration.ts
settings	persistence.ts

Carpeta	Archivos principales
`legal`	`documents.ts`, `consent.ts`
`about`	`content.ts`, `about-view.tsx`
`profile`	`constants.ts`

23.4 Estado, tema y librería

- **`src/store/`**: `store.ts`, `actions.ts`, `hooks.ts`, `slices/` (9 archivos).
- **`src/theme/`**: `colors.ts`, `typography.ts`, `spacing.ts`, `index.ts` (hooks `useThemeColors`, `useAppColorScheme`, `useTextScaleMultiplier`, `useReduceMotion`).
- **`src/lib/supabase.ts`**, **`src/hooks/use-color-scheme.ts`**.

23.5 Backend y extras

- **`supabase/migrations/`**: `20261108…symptom_logs` · `20261109…activity_progress` · `20261110…activity_sessions` · `20261111…caregiver_links` · `20261112…caregiver_observations_and_access` · `20261113…reminders` · `20261114…harden_links_and_account` · `20261115…doctor_links` · `20261116…activity_area_logica` · `20261117…caregiver_patient_name`.
- **`supabase/seed/`**, **`supabase/email-templates/`**.
- **`web-auth/`**: `confirmar.html`, `restablecer.html`, `index.html`, `auth.js`, `build.js`, `styles.css`, `vercel.json`.
- **`scripts/validar-paleta.mjs`**.
- **Docs:** `docs/REQUISITOS.md`, `DECISIONES.md`, `ENTREGA-FINAL.md`, `STATUS.md`, `ENTORNO.md`, `BRIEF.md`, `DESIGN.md`, `SUPABASE-CORREO.md`, `enunciado/`.
- **Evidencias:** `artifacts/t0…t11`, `artifacts/talkback/`.

---

24. Plan de estudio y lista de verificación

24.1 Plan sugerido (si tienes poco tiempo, prioriza lo marcado ★)

Sesión	Qué estudiar	Tiempo
1 ★	Secciones 0, 1, 5, 6 (visión general, stack, arquitectura)	1 h
2 ★	Sección 8 (Redux Toolkit) con el código abierto: `store.ts`, un slice, un thunk	1,5 h
3 ★	Sección 14 (accesibilidad): `themed-text.tsx`, `profile-option.tsx`, `acceder.tsx`, `validar-paleta.mjs`	1,5 h
4 ★	Sección 10 (RLS, funciones, triggers) con las migraciones abiertas	1,5 h
5	Secciones 7, 9, 11 (navegación, capa de datos, auth)	1 h
6	Sección 13 (tu módulo a fondo) y 12 (lógica y pruebas)	1,5 h
7 ★	Secciones 16, 18, 19 (decisiones, errores, límites)	45 min
8 ★	Sección 20 (banco de preguntas) en voz alta, y 21 (ejercicios) con el editor	1,5 h
Día de la entrega	Repasar 0.3, 21.1 y tu checklist de 17.4; **ensayar la demo**	1 h

24.2 Lista de verificación («¿puedo…?»)

- ☐ Explico la arquitectura en 1 minuto con los nombres de carpetas y capas.
- ☐ Digo qué usamos para el estado, dónde está y por qué.
- ☐ Explico `createAsyncThunk` (pending/fulfilled/rejected, `rejectWithValue`, `condition`, `requestId`).

- [ ] Sigo el recorrido de «registrar un síntoma» desde el toque hasta la base de datos.
- [ ] Explico RLS, `using` vs. `with check`, `security definer` y un trigger.
- [ ] Respondo «¿por qué no hay backend propio?» con la respuesta modelo de 10.12 g) y su trade-off.
- [ ] Explico `LargeSecureStore` y por qué existe.
- [ ] Explico la máquina de estados del consentimiento y quién puede cada transición.
- [ ] Explico `accessibilityRole/Label/Hint/State/LiveRegion` con ejemplos reales del repo.
- [ ] Explico el problema del `lineHeight` y mi solución.
- [ ] Explico cómo se movió el foco al campo inválido.
- [ ] Explico la fórmula de contraste y los umbrales.
- [ ] Digo el porqué de cada tecnología del stack.
- [ ] Nombro 5 decisiones (D-00X) con su razón.
- [ ] Cuento 5 errores reales con causa y arreglo.
- [ ] Digo 4 limitaciones **antes** de que me las pregunten.
- [ ] Abro cualquier archivo de la sección 21.1 en menos de 20 segundos.
- [ ] Hago un cambio pequeño (sección 21.2) sabiendo qué más tocar.
- [ ] (Pendiente de equipo) Tengo al menos un commit propio en mi módulo.

> **Última recomendación:** en la defensa, ante una pregunta que no recuerdas **no inventes**: di «eso está en `<archivo>`, lo abro» y léelo. Abrir el archivo correcto y explicar **lo que ves** vale más que una respuesta vaga.